

Learning Erlang from Claude

A readable companion to the official documentation · checked against OTP 29.1.1

Anatoly Rodionov, with Claude

Contents

[Introduction](#)

[1. Supervision Trees](#)

[2. Processes and Mailboxes](#)

[3. Binaries and Memory](#)

[4. ETS](#)

[5. gen_server and gen_statem](#)

[6. Distribution](#)

[7. Applications and Releases](#)

[8. Testing](#)

[9. Observability](#)

[10. Schedulers and Performance](#)

[11. NIFs and Ports](#)

[12. Elixir Alongside Erlang](#)

[13. gen_statem in Depth](#)

[14. Review Cheat Sheet](#)

Introduction

Learning Erlang from Claude · checked against OTP 29.1.1 · Sep 25, 2026

Why this book exists

AI assistants now write a large share of everyday code. They are very good at the parts a traditional textbook spends most pages on: syntax, library signatures, boilerplate. What they don't reliably get right is how the runtime actually behaves: what happens to a message nobody matches, which process owns a table, when a binary is copied, what a timeout really means.

That shifts what a programmer needs to know. Less reference detail, but not less *mechanism* detail. You can only judge generated code as well as you understand what runs underneath it. This book is built on that idea.

The official Erlang/OTP documentation is accurate and current, but written as reference: precise, spread across many guides, and hard to read from start to finish. This companion reorganises it around concepts and mechanisms, and adds the one thing reference documentation never includes: code that looks right and isn't.

How each chapter is built

Every chapter has the same five parts:

1. **Why it exists.** The concept, and the problem it solves.
2. **The building blocks.** The API, condensed into tables.
3. **How the runtime actually does it.** The mechanics, from the Reference Manual and Efficiency Guide.
4. **Plausible but wrong.** Code that compiles and passes a happy-path test, why it fails under crashes, timing or load, and the fix.
5. **Review checklist and sources.** What to check in real code, with links to the docs it came from.

Chapter 13 adds a worked example before the runtime section, so its “Plausible but wrong” part is section 5. The cheat sheet (chapter 14) follows its own layout.

The “Plausible but wrong” section is the heart of each chapter. Every example there is the kind of code an assistant (or a tired human) produces readily. Learning to spot them is the skill this book is trying to build.

How to read it

The chapters build on each other, but not strictly.

Part	Chapters	Read when
Foundations	1 Supervision Trees · 2 Processes and Mailboxes	First, in either order
Data	3 Binaries and Memory · 4 ETS	After chapter 2
Building systems	5 <code>gen_server</code> and <code>gen_statem</code> · 6 Distribution · 7 Applications and Releases	After the foundations
Keeping it working	8 Testing · 9 Observability · 10 Schedulers and Performance	Any time after chapter 5
Edges	11 NIFs and Ports · 12 Elixir Alongside Erlang	When you need them
Deep dive	13 <code>gen_statem</code> in Depth	After chapter 5
Reference	14 Review Cheat Sheet	When reviewing code

If you know Elixir but not Erlang, start with chapter 12 section 2 for the mapping between the two, then read from chapter 1. Everything about the runtime applies to both languages unchanged.

Using it with an AI assistant

Some ways this book is meant to be used:

- **As a review list.** Chapter 14 collects every checklist. Run generated code past it before accepting it.
- **As a prompt.** Ask the assistant to check its own code against a specific chapter’s “Plausible but wrong” section, or to write a test that would expose a given failure (chapter 8 shows how).
- **As a check on the assistant.** When an assistant claims something about runtime behaviour, the sources at the end of each chapter point to where you can verify it.

Conventions

- Memory sizes are in *words*: 8 bytes on a 64-bit system.
- Facts were checked against the Erlang/OTP 29.1.1 documentation, and the Elixir documentation for chapter 12. Version-specific behaviour is marked where it matters (for example “since OTP 24”).
- Code examples are deliberately short. They show one mistake each, not complete programs.

A note on accuracy

This book was written with Claude, an AI assistant, and every chapter was checked against the official documentation it cites. It can still contain mistakes, which would be fitting given its subject. Treat the linked documentation as the authority, and please report errors as issues on the repository.

1. Supervision Trees

Sample chapter · checked against OTP 29.1.1 · Sep 25, 2026

1. Why supervision exists

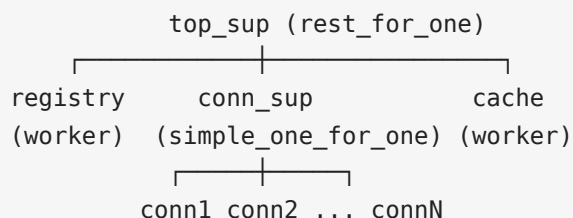
A supervisor turns an unknown crash into a known state: it restarts a failed process from a clean initial state instead of trying to repair it. That is the whole idea; everything else is configuration.

Most bugs that survive testing are transient: a race, a timeout, an odd input, a dependency briefly down. Defensive code tries to anticipate each one and usually handles them badly. Erlang inverts this. A worker handles only the cases it understands and crashes on the rest. The supervisor, which does nothing but watch, restarts it.

This works only because Erlang processes share no memory. A crash cannot corrupt a neighbour's state, so discarding one process and starting a fresh one is safe. In a shared-memory language, the same restart would leave you guessing what the dead thread left half-written.

The error kernel

Arrange processes so that the parts which must not fail are small and simple, and the risky parts sit at the leaves. Supervisors form the tree; workers do the real work. Put state you cannot afford to lose near the root, or outside process memory entirely (ETS owned by a stable process, a database, disk).



Read it left to right as a dependency order: the registry starts first because everything after it relies on it. That ordering is what `rest_for_one` exploits, as section 2 shows.

What “let it crash” does not mean

It does not mean ignoring errors. Expected failures, such as a user typing a bad password or a file not existing, are ordinary return values. You crash on the unexpected: states your code has no correct answer for. A restart only helps if the fresh state has a real chance of succeeding; a process that crashes on every start just burns the restart budget and takes its supervisor down.

2. The building blocks

A supervisor is fully described by one callback, `init/1`, returning `{ok, {SupFlags, ChildSpecs}}`. Both are maps; every key except `id` and `start` has a default, so a minimal spec is two lines.

```
init([]) ->
  SupFlags = #{strategy => rest_for_one, intensity => 5, period => 30},
  Children = [
    #{id => registry, start => {registry, start_link, []}},
    #{id => conn_sup, start => {conn_sup, start_link, []}, type => supervisor},
    #{id => cache, start => {cache, start_link, []}, shutdown => 2000}
  ],
  {ok, {SupFlags, Children}}.
```

Restart strategies: who dies with whom

The strategy encodes dependencies between siblings. Children start left to right and are terminated right to left.

Strategy	When one child dies	Use when
one_for_one (default)	Only that child restarts	Siblings are independent
rest_for_one	It and every child started after it restart	Later children depend on earlier ones
one_for_all	All children restart	Siblings share state and cannot run without each other
simple_one_for_one	Only that child restarts	Many dynamic instances of one child type

In the example above, if `registry` dies, `conn_sup` and `cache` restart too, because they may hold references to the old `registry`. If `cache` dies, nothing else is touched.

Restart types: does this child come back?

restart	Restarted after	Typical child
permanent (default)	Any exit	Long-lived servers
transient	Abnormal exit only: not <code>normal</code> , <code>shutdown</code> , or <code>{shutdown, Term}</code>	A job that should finish once but retry on failure
temporary	Never, not even when a sibling's death takes it down	Fire-and-forget tasks, per-request processes

A temporary child's spec is deleted as soon as it exits, so `restart_child/2` cannot bring it back.

Restart intensity: when the supervisor gives up

If more than `intensity` restarts happen within `period` seconds, the supervisor kills all its children and exits with reason `shutdown`. Its parent then applies its own strategy. The defaults are 1 restart per 5 seconds.

Think of the two numbers as a burst size and a sustained rate: $\text{burst} = \text{intensity}$, $\text{sustained rate} \approx \text{intensity} / \text{period}$.

So 5/30 tolerates a burst of 5 but at most one restart per 6 seconds long-term, while 1/6 has the same sustained rate but cannot survive two crashes in a row. Restarts also multiply down the tree: a child under two supervisors each allowing 10 restarts can be restarted about 100 times before the application dies.

Shutdown: how a child is stopped

shutdown	What the supervisor does	Default for
<code>brutal_kill</code>	Sends <code>kill</code> immediately	None
Integer (ms)	Sends <code>shutdown</code> , waits, then sends <code>kill</code>	Workers: 5000
<code>infinity</code>	Sends <code>shutdown</code> , waits forever	Supervisors

A child supervisor must use `infinity`; a finite value can kill it midway through stopping its own children. For a worker, `infinity` makes the whole tree's shutdown depend on that worker's `terminate/2` always returning.

Automatic shutdown (OTP 24+)

A supervisor can represent one unit of work and stop itself when that work is done. Set `auto_shutdown => any_significant` or `all_significant`, and mark the relevant `transient` or temporary children `significant => true`. The children just exit normally; they need no knowledge of the tree. Keep such supervisors non-permanent in their parent, or the parent will restart them straight back into shutting down.

3. How the runtime actually does it

Supervisors are ordinary Erlang code built on three VM primitives: links, exit signals, and trapping exits. Knowing these explains every surprising behaviour in section 4.

Links and exit signals

A link is a two-way bond between processes. When one dies, the VM sends an exit signal carrying its exit reason to every linked process. What the receiver does depends on the reason and on whether it traps exits:

Signal reason	Receiver not trapping exits	Receiver trapping exits
<code>normal</code>	Ignored	Gets <code>{'EXIT', From, normal}</code> message
Any other reason	Dies with the same reason	Gets <code>{'EXIT', From, Reason}</code> message
<code>kill</code> (sent explicitly)	Dies with reason <code>killed</code>	Dies with reason <code>killed</code> : cannot be trapped

A supervisor traps exits. That single flag is what turns a child's death from something that kills the supervisor into a message it can act on.

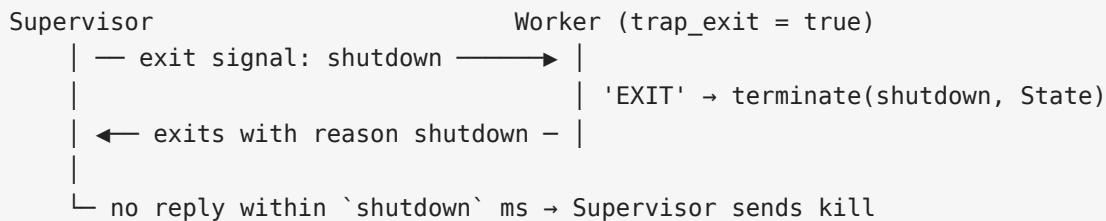
Starting: synchronous and in order

Each child's start function must spawn, link, and return `{ok, Pid}`. The OTP `start_link` functions do this through `proc_lib`, which blocks the caller until the child's `init/1` returns. Consequences:

- Children start strictly in list order; child 2's `init/1` can rely on child 1 being up.
- A slow `init/1` blocks the supervisor, and so the whole boot sequence.
- If any child fails to start at boot, the supervisor stops the ones already running and fails with `{error, {shutdown, Reason}}`.
- Returning `ignore` from `init/1` is allowed: the spec is kept (unless temporary) with no process behind it.

Restarts and `terminate_child/2` also run inside the supervisor process. While one is in progress, the supervisor answers nothing else.

Stopping: the shutdown protocol



The supervisor sends `shutdown`, then waits for the link's `'EXIT'` back, and escalates to `kill` on timeout. Children are stopped in reverse start order; `simple_one_for_one` children are stopped in parallel, in no defined order.

The detail people miss: a `gen_server` only runs `terminate/2` on shutdown if it traps exits. Without `process_flag(trap_exit, true)` in `init/1`, the `shutdown` signal kills it directly, and cleanup code never runs. With `brutal_kill`, `terminate/2` never runs either way.

Why restarts are safe

A restarted child is a new process with a new pid and a state built from scratch by `init/1`. Nothing carries over except what lives outside the process: registered names, ETS tables owned by others, external systems. Anyone holding the old pid now holds a dead reference, which is why sections 2 and 4 keep pointing at dependency order and names.

4. Plausible but wrong

Each snippet below compiles, passes a happy-path test, and is the kind of code an assistant produces readily. Each fails only under a crash or a shutdown.

4.1 Cleanup that never runs

```
init(Path) ->
  {ok, Fd} = file:open(Path, [append]),
  {ok, #{fd => Fd}}.

terminate(_Reason, #{fd := Fd}) ->
  file:sync(Fd), file:close(Fd).
```

On shutdown the supervisor sends an exit signal, not a message. The process does not trap exits, so it dies instantly and `terminate/2` is skipped; buffered data is lost. Fix: call `process_flag(trap_exit, true)` first in `init/1`, and give the child an integer `shutdown`, not `brutal_kill`.

4.2 A supervisor child that forgot its type

```
#{id => conn_sup, start => {conn_sup, start_link, []}}
```

`type` defaults to `worker`, and a worker's `shutdown` defaults to 5000 ms. A busy subtree that needs longer is killed mid-shutdown and can leave orphaned grandchildren. Fix: `type => supervisor`, which also makes `shutdown` default to `infinity`.

4.3 Connecting in `init/1`

```
init(Opts) ->
  {ok, Conn} = db:connect(Opts),    % may take seconds, may fail
  {ok, #{conn => Conn}}.
```

`init/1` blocks the supervisor and every sibling after it. If the database is down, the child crashes on each start and exhausts the default 1 restart per 5 s almost immediately, taking the supervisor, then its parent, then the application down. A dependency outage becomes a node outage. Fix: return quickly with `{ok, State, {continue, connect}}`, connect in `handle_continue/2`, and retry with backoff while reporting “not ready” to callers.

4.4 Holding a sibling's old pid

```
%% cache.erl, under a one_for_one supervisor
init([]) ->
  Reg = whereis(registry),
  {ok, #{registry => Reg}}.
```

When `registry` restarts it gets a new pid. `cache` keeps sending to the dead one: `gen_server:call` exits with `noproc`, and plain `!` fails silently. Fix: address it by registered name on every call, or monitor it, or encode the dependency with `rest_for_one` so `cache` restarts after `registry`.

4.5 Permanent per-connection workers

```
init([]) ->
    {ok, [{strategy => simple_one_for_one},
          [{id => conn, start => {conn, start_link, []}}]}.

```

The child defaults to `permanent` and the supervisor to 1 restart per 5 s. Two clients sending bad input within 5 seconds kill the supervisor and every healthy connection with it. A restarted connection process has no socket anyway. Fix: `restart => temporary` for per-request or per-connection processes; let the client reconnect.

4.6 Stopping your own supervisor

```
handle_info(job_done, State) ->
    supervisor:terminate_child(parent_sup, my_sup),
    {noreply, State}.

```

A process inside `my_sup` asks the parent to stop `my_sup`. The parent waits for the subtree to exit; the subtree waits for this process; this process waits for the call to return. If this process traps exits, it deadlocks until its shutdown timeout fires; if not, it is killed mid-call. Fix: mark the process `significant` in a supervisor with `auto_shutdown`, and just return `{stop, normal, State}`.

5. Review checklist and sources

Use this when reviewing a supervisor, whether you wrote it or an assistant did.

- ☐ Child order matches dependency order, and the strategy reflects it (`rest_for_one` where later children need earlier ones)
- ☐ Every child supervisor has `type => supervisor`
- ☐ Workers with cleanup trap exits and have an integer `shutdown`
- ☐ `init/1` returns fast; slow setup is in `handle_continue/2`
- ☐ Per-request and per-connection children are `temporary`
- ☐ `intensity/period` are set deliberately: burst size and sustained rate, not identical at every level
- ☐ No process holds a sibling's pid across a possible restart
- ☐ Work-unit supervisors use `auto_shutdown` and are not `permanent` in their parent

Sources

- Supervisor Behaviour, OTP Design Principles: https://www.erlang.org/doc/system/sup_princ.html
- supervisor module reference, stdlib: <https://www.erlang.org/doc/apps/stdlib/supervisor.html>

2. Processes and Mailboxes

Chapter 2 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why processes are the unit of everything

An Erlang process is a tiny isolated computer: its own heap, its own mailbox, and no way to touch anyone else's memory. Everything else in OTP, supervision included, is built on that isolation.

Processes are cheap enough to use freely. A new process takes 327 words of memory, 233 of them for its initial heap and stack. A million idle processes fit in a few gigabytes, so “one process per connection, per request, per game session” is the normal design, not an extravagance.

The price of isolation is copying. Sending a message copies the term into the receiver's heap, with two exceptions on the same node: large (refc) binaries and literals are shared, not copied. This is what lets a crashed process vanish without corrupting anyone: nobody else holds pointers into its memory.

Share nothing, but pay per send

The mental model is postal, not shared-memory. You hand over a copy; you never get a lock. That makes reasoning local and garbage collection per-process (no global pauses), but it also means sending a 10 MB map to 100 processes costs 1 GB of copying.

2. The building blocks

Primitive	What it does	Direction	Notes
<code>spawn/1,3</code>	Starts a process	—	Failure goes unnoticed unless you link or monitor
<code>link/1, spawn_link</code>	Ties two lifetimes together	Two-way	One link per pair; repeated <code>link</code> has no effect
<code>monitor(process, Pid)</code>	Watches a process	One-way	Each call is a separate monitor; each sends its own <code>'DOWN'</code>
<code>register(Name, Pid)</code>	Gives a pid an atom name	—	Removed automatically when the process dies
<code>alias/0,1</code>	A reference that can receive messages	—	Lets a caller refuse late replies (used by <code>gen_server:call</code>)
<code>Pid ! Msg</code>	Sends a message	—	Never fails for a dead pid; the message is dropped

Links versus monitors

Use a link when two processes should live and die together; use a monitor when one only needs to know the other died. A monitor on a pid that is already dead delivers `{'DOWN', Ref, process, Pid, noproc}` at once, so there is no race between checking and watching.

The only ordering guarantee

If process A sends S1 and then S2 to B, S1 will not arrive after S2. That is all. There is no ordering across different senders and no causal ordering through intermediaries:

```
A — m1 —————> B
A — m2 —> C — m2 (forward) —> B    B may see m2 before m1
```

Receive

`receive` walks the mailbox from the oldest message, trying the clauses top to bottom against each message. The first message that matches any clause is removed; every other message stays where it was. `after T` fires if nothing matched within T milliseconds; `after 0` checks only what is already there.

3. How the runtime actually does it

The mailbox is a queue you scan

Each `receive` is $O(N)$ in the number of messages ahead of the one it matches. Unmatched messages are not errors; they just sit there and are rescanned on every later `receive`. In a busy process, a message nobody matches turns every `receive` into a longer walk, forever.

The reference trick

There is one pattern the compiler and runtime make cheap. If you create a reference and every clause of the next `receive` matches on it, the runtime remembers the queue position at creation and scans only messages that arrived after it:

```
call(Pid, Req) ->
  Ref = monitor(process, Pid),
  Pid ! {request, self(), Ref, Req},
  receive
    {reply, Ref, Result} ->
      demonitor(Ref, [flush]),
      Result;
    {'DOWN', Ref, process, Pid, Reason} ->
      exit(Reason)
  after 5000 ->
    demonitor(Ref, [flush]),
    exit(timeout)
end.
```

Compile with `erlc +recv_opt_info` to see whether a given `receive` got the optimization. `gen_server:call` uses this pattern, plus an alias (since OTP 24) so a reply arriving after the timeout never lands in the caller's mailbox.

Where messages live

By default, messages sit on the receiving process's heap and are scanned by its garbage collector. A process that receives heavy traffic can set `process_flag(message_queue_data, off_heap)` so queued messages live outside the heap and stop inflating GC work.

Memory controls

Tool	What it does	Use when
<code>min_heap_size</code> (in <code>spawn_opt</code>) or <code>+h</code>	Larger starting heap, fewer early GCs	Few processes, measured benefit only
<code>erlang:hibernate/3</code> or <code>hibernate return</code>	Shrinks the process to minimum memory until the next message	Long-idle processes
<code>max_heap_size</code> flag	Kills and/or logs a process whose heap exceeds a limit	Guarding against runaway mailboxes or state

The docs warn that a larger initial heap also means fewer collections, which can keep large binaries alive longer than you want. Measure before tuning.

4. Plausible but wrong

4.1 A receive loop with no catch-all

```
loop(State) ->
  receive
    {add, X}    -> loop(State + X);
    {get, From} -> From ! State, loop(State)
  end.
```

Any other message (a stray reply, an `'EXIT'`, a typo like `{Add, 1}`) stays in the mailbox forever. Each loop iteration rescans it, so the process slows down and grows without ever failing. Fix: add a final clause that logs and drops unknown messages, or use `gen_server`, whose `handle_info` makes the catch-all explicit.

4.2 Waiting for a reply without a fresh reference

```
Pid ! {request, self(), Req},
receive
  {reply, Result} -> Result
end
```

With a busy mailbox this scans every queued message each time, which turns into quadratic behaviour under load. It can also pick up the wrong reply: one left over from an earlier request that timed out. Fix: tag each request with a new reference (or monitor) and match on it, as in the `call/2` example above.

4.3 Assuming ordering through a third process

```
%% in A
B ! {config, NewConfig},
C ! {apply_and_notify, B}.    %% C then sends B 'config_applied'
```

B may receive `config_applied` before `{config, ...}`. Only direct A→B order is guaranteed. Fix: route both messages through the same sender, or have B wait for the config message explicitly before acting on the notification.

4.4 Sending to a name that may not exist

```
notify(Event) ->
  logger_proc ! {event, Event}.
```

Sending to a pid never fails, but sending to an unregistered name raises `badarg`. While `logger_proc` is restarting, every caller of `notify/1` crashes. Fix: decide what should happen during the gap. Either let the caller crash knowingly, or use `whereis/1` and handle `undefined`, or go through a supervisor-owned API that absorbs restarts.

4.5 Retrying a timed-out call

```
charge(Card, Amount) ->
  try gen_server:call(payments, {charge, Card, Amount}, 2000)
  catch exit:{timeout, _} -> charge(Card, Amount)
  end.
```

Since OTP 24 the late reply is discarded safely, but the server may still have done the work. A timeout means “I stopped waiting”, not “it didn’t happen”. This code can charge twice. Fix: make the operation idempotent (a request id the server deduplicates), or treat the timeout as an unknown outcome and check state before retrying. Note also that a `gen_server` calling itself exits with `calling_self` rather than deadlocking.

4.6 Fan-out of a large term

```
[Pid ! {snapshot, BigMap} || Pid <- Workers]
```

Each send copies `BigMap` into another heap. With 1,000 workers and a 5 MB map, that is 5 GB of copying and 1,000 extra GC loads. Fix: put shared read-mostly data in ETS (readers copy only the rows they need), or send a key and let workers look it up. Large binaries are the exception: those over 64 bytes are shared by reference on the same node.

4.7 Fire-and-forget spawn

```
handle_cast({resize, Img}, State) ->
    spawn(fun() -> resize_and_store(Img) end),
    {noreply, State}.
```

If `resize_and_store/1` crashes, nobody knows; the image is just missing. If the parent stops, the worker keeps running unsupervised. Fix: start it under a `simple_one_for_one` (or `Task`) supervisor, or `spawn_monitor` it and handle the `'DOWN'` result.

5. Review checklist and sources

- ☐ Every hand-written `receive` loop has a catch-all clause or is replaced by a behaviour
- ☐ Every request/reply is tagged with a fresh reference or monitor, and matched on it
- ☐ No code relies on ordering except between one sender and one receiver
- ☐ Sends to registered names handle the “not registered right now” case
- ☐ Timeouts are treated as unknown outcomes; retried operations are idempotent
- ☐ Large shared data is in ETS or passed as binaries, not copied per send
- ☐ Every spawned process is linked, monitored, or supervised
- ☐ High-traffic processes have `message_queue_data` considered; heap tuning is measured

Sources

- Processes, Efficiency Guide: https://www.erlang.org/doc/system/eff_guide_processes.html
- Processes, Reference Manual: https://www.erlang.org/doc/system/ref_man_processes.html
- Expressions (receive), Reference Manual: <https://www.erlang.org/doc/system/expressions.html>
- `gen_server` module reference: https://www.erlang.org/doc/apps/stdlib/gen_server.html
- `erlang` module reference: <https://www.erlang.org/doc/apps/erts/erlang.html>

3. Binaries and Memory

Chapter 3 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why memory layout matters in Erlang

In Erlang, the way you represent data decides both its size and whether a message copies it. The same 1 KB of text can cost 1 KB or 16 KB, and it can be shared between processes or copied into each one, depending only on the type you picked.

Sizes are measured in words: 8 bytes on a 64-bit system (check with `erlang:system_info(wordsize)`).

Term	Size
Small integer, atom, local pid	1 word
Float (64-bit)	3 words
Tuple	2 words + elements
List	1 word + 1 per element + elements
String as a list of characters	1 word + 2 per character
Small map (≤ 32 keys)	5 words + keys and values
Binary	3-6 words + the bytes

So `"hello world"` as a character list costs 23 words (184 bytes). As a binary it is 11 bytes plus a small header. Elixir strings are already UTF-8 binaries; charlists (`~c"..."`) are the list form and cost the same as Erlang strings.

2. The building blocks

Four kinds of binary

Internally the runtime has four binary representations. You never choose between them directly, but knowing which one you have explains most binary memory surprises.

Kind	Where the bytes live	When you get it
Heap binary	On the process heap	Up to 64 bytes; copied on send like any term
Refc binary	Outside all heaps, reference-counted	Over 64 bytes; shared on send, only a small handle is copied
Sub-binary	Points into another binary	Slicing or matching out part of a binary

Kind	Where the bytes live	When you get it
Match context	Internal pointer used during matching	Binary pattern matching in loops

The docs note that binary handling was rewritten in OTP 27 and the Efficiency Guide text has not fully caught up. The internal names may shift, but the behaviour this chapter relies on is the same.

iodata: text you never have to concatenate

`iodata()` is a binary, or a list that nests bytes, binaries and more `iodata`. Files, sockets and ports accept it directly, so you can build output as a tree and skip concatenation entirely:

```
Html = [<<"<li>">>, Name, <<"</li>">>],
ok = gen_tcp:send(Socket, [Header, Html, Footer]).
```

Call `iolist_to_binary/1` only when you really need one flat binary, such as a hash input or a map key.

Atoms are forever

The atom table is not garbage-collected. By default it holds 1,048,576 atoms (change with `+t`), and each atom can be at most 255 characters. Atoms are ideal for a fixed vocabulary of tags and names, and dangerous for anything derived from input.

3. How the runtime actually does it

Refc binaries and their handles

A large binary is two pieces: the bytes, stored off-heap with a reference count, and a small handle (a `ProcBin`) on each process heap that uses it. The count drops only when a handle is garbage-collected. So a large binary is freed only after every process that ever touched it has run a garbage collection since dropping it.

Sub-binaries keep the whole parent alive

Slicing a large binary usually does not copy. A slice larger than 64 bytes points into the original, so a 1 KB slice of a 10 MB binary keeps all 10 MB alive for as long as the slice exists. Slices of 64 bytes or less are copied into small heap binaries instead; tested on OTP 25, a 64-byte slice referenced 64 bytes while a 65-byte slice referenced all 10 MB. `binary:referenced_byte_size/1` shows how much a binary is really holding onto.

The append optimization

Building a binary with `<<Acc/binary, More/binary>>` in a loop is efficient: the runtime over-allocates and appends in place. That only works while there is a single handle with a single reference to the accumulator. The following force a copy on the next append:

- keeping and using an older version of the accumulator
- sending the accumulator in a message
- inserting it into ETS
- passing it to a port
- pattern matching on it

The match optimization

Recursive matching like `<<H, T/binary>>` is compiled to reuse one match context instead of creating a sub-binary per step. Compile with `erlc +bin_opt_info` to see where the optimization applied and where it didn't.

4. Plausible but wrong

4.1 Prepending to a binary accumulator

```
encode([], Acc) -> Acc;
encode([X | Xs], Acc) -> encode(Xs, <<(esc(X))/binary, Acc/binary>>).
```

Prepending cannot use the append optimization, so every step copies the whole accumulator: $O(n^2)$ for n items. It looks exactly like the efficient version. Fix: append (`<<Acc/binary, (esc(X))/binary>>`) over reversed input, or collect an iolist and flatten once at the end.

4.2 Quadratic string building with ++

```
render(Rows) ->
  lists:foldl(fun(R, Acc) -> Acc ++ format_row(R) end, "", Rows).
```

`++` copies its whole left operand, so this is $O(n^2)$, and the result is a character list at 16 bytes per character. Fix: `[format_row(R) || R <- Rows]` is already valid iodata; hand it straight to the socket or file.

4.3 Keeping a small slice of a huge binary

```
handle_info({http_body, Body}, State) -> %% Body is 10 MB
  <<_:32/binary, Header:512/binary, _/binary>> = Body,
  {noreply, State#{last_header => Header}}.
```

`Header` is a 512-byte sub-binary, so the process state now pins the full 10 MB for as long as it keeps it. Repeat this per request and memory climbs with no obvious cause. (A slice of 64 bytes or less would have been copied automatically, which is why a short ID doesn't show the problem.) Fix:

`binary:copy(Header)` before storing it. The docs caution that copying only helps when nothing else still references the large binary, so confirm with `binary:referenced_byte_size/1` first.

4.4 The router that never collects

```
%% long-lived process forwarding large payloads
loop(Routes) ->
  receive
    {msg, Key, Payload} ->
      maps:get(Key, Routes) ! Payload,
      loop(Routes)
  end.
```

Each payload is a large refc binary, but the router allocates almost nothing on its own heap, so it rarely garbage-collects. The handles pile up and every payload it has ever forwarded stays alive. Binary memory grows while the process itself looks small. Fix: hibernate the process when it goes idle, lower its `fullsweep_after` through `spawn_opt`, or send payloads directly so the router never touches them.

4.5 Atoms from input

```
handle_request(#{<<"action">> := Action} = Req) ->
  dispatch(binary_to_atom(Action), Req).
```

Every distinct value a client sends becomes a permanent atom. An attacker, or just a long-running system, eventually reaches 1,048,576 atoms and the node goes down. Fix: `binary_to_existing_atom/1` (fails unless the atom already exists), or match the binary directly: `dispatch(<<"create">>, Req) -> ...`.

4.6 Breaking the append optimization by accident

```
build([], Acc) -> Acc;
build([X | Xs], Acc) ->
  Acc1 = <<Acc/binary, X:32>>,
  log_size(byte_size(Acc)),      %% still uses the old Acc
  build(Xs, Acc1).
```

Using `Acc` after creating `Acc1` means two live versions of the accumulator, so the next append must copy. Fix: use only the newest version (`byte_size(Acc1)`), and check with `+bin_opt_info`.

5. Review checklist and sources

- ☐ Text in hot paths is binaries or iodata, not character lists
- ☐ Output is built as iodata and written once; no `++` or binary concatenation in loops
- ☐ Binary accumulators only ever append, and only the newest version is used
- ☐ Slices over 64 bytes of large binaries that are stored long-term are copied (after checking `referenced_byte_size`)
- ☐ Long-lived processes that touch large binaries hibernate or garbage-collect regularly
- ☐ No atoms are created from external input
- ☐ `+bin_opt_info` has been checked on binary-heavy modules

Sources

- Constructing and Matching Binaries, Efficiency Guide: <https://www.erlang.org/doc/system/binaryhandling.html>
- Memory Usage, Efficiency Guide: <https://www.erlang.org/doc/system/memory.html>
- System Limits: https://www.erlang.org/doc/system/system_limits.html
- binary module reference: <https://www.erlang.org/doc/apps/stdlib/binary.html>
- erlang module reference: <https://www.erlang.org/doc/apps/erts/erlang.html>

4. ETS

Chapter 4 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why ETS exists

ETS is shared, in-memory storage owned by a process. It breaks the “share nothing” rule of chapter 2 in a controlled way: many processes can read one table without sending messages to anyone.

A `gen_server` that holds state in its loop is a bottleneck: every read waits in one mailbox. An ETS table lets readers go straight to the data, in constant time for a `set`, with no mailbox and no serialisation through a single process. That makes it the standard tool for caches, registries, counters and lookup tables.

What it does not remove is copying. Every insert copies the object into the table and every lookup copies it back out. ETS saves you the message round-trip, not the copy.

2. The building blocks

Table types

Type	Objects per key	Lookup cost	Notes
<code>set</code> (default)	1	Constant	Hash table
<code>ordered_set</code>	1	$O(\log N)$	Tree, keys in order; <code>1</code> and <code>1.0</code> are the same key
<code>bag</code>	Many, distinct	Grows with objects under that key	Avoid many objects per key
<code>duplicate_bag</code>	Many, duplicates allowed	Same as <code>bag</code>	Same warning

Access and ownership

Option	Who reads	Who writes
<code>protected</code> (default)	Any process	Owner only
<code>public</code>	Any process	Any process
<code>private</code>	Owner only	Owner only

Every table has one owner process. When the owner dies, the table is deleted, unless it has an `heir`, which then inherits it. `ets:give_away/3` hands a table to another living process. `named_table` lets you refer to it by atom instead of by table id.

Concurrency options

Option	Helps when	Cost
<code>read_concurrency</code>	Many parallel readers, bursts of reads	Switching between reads and writes gets more expensive
<code>write_concurrency</code> (true or auto)	Many processes writing different keys	More memory and per-operation overhead
<code>decentralized_counters</code>	Frequent inserts and deletes from many schedulers	<code>ets:info(T, size)</code> and memory become slower

These are tuning knobs, not correctness switches. Turn them on for a measured workload, not by default.

persistent_term: the read-only cousin

`persistent_term` is for data read constantly and changed almost never. Reads take no locks and do not copy the term. But every `put/2` or `erase/1` of a complex term starts a global garbage collection across all processes. The docs say plainly that it is “not a general replacement for ETS tables.”

3. How the runtime actually does it

What is atomic

Every update to a single object is atomic and isolated. A reader never sees half an update, and an update either happens completely or not at all. `update_counter` and `update_element` extend this to read-modify-write on fields of one object.

Nothing larger is atomic. There are no transactions across two keys, and a lookup followed by an insert is two separate operations that another process can interleave.

Traversal is not a snapshot

No traversal gives a consistent snapshot of a table that is being updated concurrently. Walking a `set` with `first/next` while others insert can skip keys, return a key twice, or raise `badarg`. Traversal is safe if:

- the table is an `ordered_set`, or
- the whole traversal is one ETS call (such as `ets:select/2`), or
- you wrap it in `ets:safe_fixtable/2`.

Keys decide the cost of a query

`lookup/2` with a known key is constant time on a `set` and $O(\log N)$ on an `ordered_set`. `match` and `select` whose pattern leaves the key unbound have to scan the entire table. An `ordered_set` can avoid the full scan when the pattern binds a prefix of the key, such as the first element of a tuple key.

4. Plausible but wrong

4.1 A table that dies with its creator

```
init([]) ->
  ets:new(sessions, [named_table, public, set]),
  {ok, #{} }.
%% handle_call/3 crashes on some bad input...
```

The `gen_server` owns the table. The first crash deletes every session, and processes reading `sessions` during the restart get `badarg`. The table's lifetime is tied to your least reliable code. Fix: give the table a dedicated owner that does nothing else, started earlier under the same supervisor, or set an `heir`. Decide deliberately whether the data should survive a crash; sometimes a fresh table is exactly what you want.

4.2 Read, then write

```
hit(Key) ->
  N = case ets:lookup(stats, Key) of
        [{_, C}] -> C;
        []       -> 0
      end,
  ets:insert(stats, {Key, N + 1}).
```

Two processes can read the same `N` and both write `N + 1`, losing one hit. Under load this undercounts silently. Fix: `ets:update_counter(stats, Key, 1, {Key, 0})`, which increments atomically and inserts the default if the key is missing.

4.3 Querying by a field that isn't the key

```
users_in(City) ->
  ets:match_object(users, {'_', '_', City, '_'}).
```

The key is unbound, so this scans every user on every call. It's fine in a test with 50 rows and becomes the slowest thing in production. `tab2list` plus a filter is the same scan with extra copying. Fix: keep a secondary index (bag of `{City, UserId}`), or choose a key that matches the common query. With an `ordered_set` keyed `{City, UserId}`, binding `City` avoids the full scan.

4.4 Walking a busy table by hand

```
expire(T) -> expire(T, ets:first(T)).
expire(_, '$end_of_table') -> ok;
expire(T, K) ->
  maybe_delete(T, K),
  expire(T, ets:next(T, K)).
```

On a `set` that other processes are writing, this can miss keys, visit some twice, or crash with `badarg` when `K` has been deleted. Fix: wrap the loop in `ets:safe_fixtable(T, true) / false`, or replace it with one `ets:select_delete/2` call using a match spec on the expiry field.

4.5 One giant object

```
ets:insert(config, {all, ConfigMap}),           %% 2 MB map
...
[{all, Cfg}] = ets:lookup(config, all),         %% on every request
Timeout = maps:get(timeout, Cfg).
```

Every request copies 2 MB to read one integer. Fix: store one object per setting (`{timeout, 5000}`), or, for configuration that changes almost never, `persistent_term:get({myapp, timeout})`, which does not copy.

4.6 persistent_term as a cache

```
remember(UserId, Profile) ->
    persistent_term:put({profile, UserId}, Profile).
```

Every `put` that replaces a complex term triggers a global GC that scans every process in the node. Used as a per-user cache, this stalls the whole system in proportion to how busy the cache is. Fix: ETS for anything that changes; `persistent_term` only for values set at startup or on rare configuration changes.

4.7 A bag with a hot key

```
ets:new(events, [bag, named_table, public]),
ets:insert(events, {Topic, Event}).           %% one popular Topic
```

Inserts and lookups on a `bag` get slower in proportion to the number of objects under the same key. The docs add that this linear search does not yield, so it also hurts the scheduling of other processes. Fix: add a unique part to the key (`{Topic, Seq}`) in an `ordered_set` and query by prefix.

5. Review checklist and sources

- ☐ Every table's owner is chosen deliberately (dedicated process or `heir`), and crash behaviour is intended
- ☐ Counters and other read-modify-write updates use `update_counter` / `update_element`
- ☐ Frequent queries use the key; other access paths have a secondary index
- ☐ No `tab2list` or unbound-key `match/select` on large tables in hot paths
- ☐ Traversals of concurrently updated tables use `ordered_set`, one ETS call, or `safe_fixtable`
- ☐ Objects are small enough to copy on every lookup
- ☐ `persistent_term` holds only rarely changed data
- ☐ Concurrency options are enabled for a measured reason
- ☐ `bag` tables have no hot keys

Sources

- ets module reference: <https://www.erlang.org/doc/apps/stdlib/ets.html>
- Tables and Databases, Efficiency Guide: <https://www.erlang.org/doc/system/tablesdatabases.html>
- persistent_term module reference: https://www.erlang.org/doc/apps/erts/persistent_term.html

5. gen_server and gen_statem

Chapter 5 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why behaviours exist

A behaviour splits a process into a generic part that OTP writes once and a specific part you write as callbacks. `gen_server` and `gen_statem` are the two you will use most. The generic part is where the hard things live: the start handshake with the supervisor, system messages, debugging and tracing through `sys`, error reports, code upgrade, and the shutdown protocol.

A hand-written `receive` loop gets none of that. It looks simpler, but it cannot be suspended, inspected, traced or upgraded like the rest of the system. The rule of thumb is short: if a process lives longer than one task, make it a behaviour.

Both behaviours are still just one process. Every callback runs in sequence inside it, so a behaviour is also a serialisation point: whatever one callback is doing, every other request waits.

In Elixir, `GenServer` wraps `gen_server`. `:gen_statem` is usually used directly from Elixir; the concepts below carry over unchanged.

2. The building blocks

gen_server callbacks

Callback	Triggered by	Typical return
<code>init/1</code>	<code>start_link</code>	<code>{ok, State}</code> or <code>{ok, State, {continue, Term}}</code>
<code>handle_call/3</code>	<code>gen_server:call</code> (caller waits)	<code>{reply, Reply, State}</code> or <code>{noreply, State}</code>
<code>handle_cast/2</code>	<code>gen_server:cast</code> (caller does not wait)	<code>{noreply, State}</code>
<code>handle_info/2</code>	Any other message: timers, <code>'DOWN'</code> , <code>'EXIT'</code>	<code>{noreply, State}</code>
<code>handle_continue/2</code>	A <code>{continue, Term}</code> return	<code>{noreply, State}</code>
<code>terminate/2</code>	Stop, or shutdown when trapping exits	ignored

If you don't export `handle_info/2`, the default logs unexpected messages and drops them.

gen_statem in one table

A `gen_statem` separates the *state* (which mode the machine is in) from the *data* (everything else). Events arrive with a type:

Event type	Comes from
<code>{call, From}</code>	<code>gen_statem:call</code>
<code>cast</code>	<code>gen_statem:cast</code>
<code>info</code>	Ordinary messages
<code>internal</code>	Your own <code>{next_event, internal, E}</code> action
<code>state_timeout</code>	A timer that dies on any state change
<code>{timeout, Name}</code>	A named timer that survives state changes
<code>timeout</code>	An event timer that dies on <i>any</i> event

There are two callback modes:

- `state_functions`: one function per state, so states must be atoms. Easy to read when states are few and simple.
- `handle_event_function`: one `handle_event/4` for everything, so states can be any term, such as `{connected, Retries}`.

The actions that make `gen_statem` worth using:

- **postpone**: put this event back and retry it after the next state change.
- **next_event**: inject an event to be handled before anything else in the mailbox.
- **state enter calls**: run code on every entry into a state, in one place instead of at every transition.

Which one?

Use <code>gen_server</code> when	Use <code>gen_statem</code> when
The process is a service: a cache, a pool, a registry	Behaviour depends on a mode: connecting, connected, backing off
Every request is handled the same way whatever the state	Some events are only valid in some states
Timeouts are simple or absent	You need per-state timers or events that wait for the right state

The docs put the overhead of `gen_statem` over `gen_server` as marginal, so choose by shape, not speed. Chapter 13 works through a complete `gen_statem` in depth.

3. How the runtime actually does it

A call is a monitored request

`gen_server:call` is the pattern from chapter 2: monitor the server, send a request tagged with a reference, and wait for a reply matching it. The default timeout is 5000 ms. If the server dies, the caller exits with the server's reason (or `noproc` if it was never there). Since OTP 24 an alias ensures that a reply arriving after the timeout is discarded.

Replies do not have to be immediate

`handle_call` may return `{noreply, State}` and answer later with `gen_server:reply(From, Reply)` from any later callback, or from another process. This is how a server stays responsive while slow work happens elsewhere.

Timeouts in `gen_server` are crude

`{noreply, State, Timeout}` sends `timeout` to `handle_info` only if *no message at all* arrives for `Timeout` ms. Any message cancels it, and the docs admit that even a system message restarts it ("a known and unfortunate flaw"). For real timers, use `erlang:send_after/3` or `erlang:start_timer/3` and keep the reference.

Timeouts in `gen_statem` are precise

Timer	Cancelled by	Good for
<code>timeout</code> (event)	Any event	"Nothing at all happened for N ms"
<code>state_timeout</code>	Leaving the state	"Give up on connecting after 10 s"
<code>{timeout, Name}</code>	Only by you (or it fires)	Timers that span states, several at once

How postpone works

A postponed event is set aside and retried after the next *state change*, meaning `NewState /= OldState`. Changing only the data does not retry anything. The docs spell out the consequence: if a value decides which events you can handle, it belongs in the state, not the data.

4. Plausible but wrong

4.1 Slow work inside `handle_call`

```
handle_call({report, Id}, _From, State) ->
    Report = build_report(Id),           %% 20 seconds
    {reply, Report, State}.
```

For those 20 seconds every other caller is stuck, and anyone using the default 5000 ms timeout crashes with `timeout`. The server looks hung while it is actually busy. Fix: hand the work to a separate process and reply from there:

```
handle_call({report, Id}, From, State) ->
    spawn_link(fun() -> gen_server:reply(From, build_report(Id)) end),
    {noreply, State}.
```

In production, start the worker under a task supervisor instead of `spawn_link` (chapter 2, section 4.7).

4.2 An idle timeout that never fires

```
handle_info(tick, State) -> {noreply, State, 60000};
handle_info(timeout, State) -> {stop, normal, State}.
%% a metrics collector sends 'tick' every 10 s
```

Every message restarts the 60-second timeout, so a process meant to stop after a minute of real inactivity lives forever. Fix: track the time of the last meaningful event and use an explicit timer (`erlang:start_timer/3`), or use `gen_statem` with a named timeout that only meaningful events reset.

4.3 A state machine hidden in a gen_server

```
handle_call({send, Msg}, _From, #{status := connecting} = S) ->
    {reply, ok, S#{pending := [Msg | maps:get(pending, S)]}};
handle_info(connected, #{pending := P} = S) ->
    [do_send(M) || M <- lists:reverse(P)],
    {noreply, S#{status := connected, pending := []}}.
```

Each status adds more clauses, manual queues and chances to lose a message on a path you forgot. Fix: make it a `gen_statem` where `send` is postponed in `connecting` and replayed automatically on entering `connected`.

4.4 Postponing on data, not state

```
handle_event({call, _}, {send, _}, open, #{credit := 0}) ->
    {keep_state_and_data, [postpone]};
handle_event(info, {credit, N}, open, D) ->
    {keep_state, D#{credit := N}}.
```

The state stays `open`, so the postponed sends are never retried, even after credit arrives. The callers hang until they time out. Fix: make credit part of the state, such as `{open, Credit}` in `handle_event_function` mode, or move between states `open` and `blocked`.

4.5 Using an event timeout for a per-state limit

```
connecting(enter, _Old, D) ->
    {keep_state, D, [{timeout, 10000, give_up}]};
```

```
connecting(info, {status_query, From}, D) ->
  From ! connecting,
  keep_state_and_data.
```

An event timeout is cancelled by *any* event, so the first status query cancels it. Under monitoring that polls every few seconds, the connection attempt never gives up. Fix: `{state_timeout, 10000, give_up}`, which only a change of state cancels.

4.6 Two servers calling each other

```
%% in a:handle_call(get_total, ...)
Total = gen_server:call(b, sum),
%% in b:handle_call(sum, ...)
Rate = gen_server:call(a, rate),
```

A waits for B, B waits for A. Nothing moves until the 5-second timeout, then one or both crash. It only happens when the two requests overlap, so it rarely shows up in tests. Fix: make one direction a `cast` or a deferred reply, or restructure so data flows one way only.

4.7 Casting without backpressure

```
log(Line) -> gen_server:cast(log_writer, {line, Line}).
```

When producers outpace the writer, its mailbox grows without limit and memory climbs until the node dies. A `call` naturally slows producers to the writer's speed. Fix: use `call` for high-volume paths, or add explicit batching or load shedding.

5. Review checklist and sources

- ☐ Long-lived processes are behaviours, not hand-written receive loops
- ☐ No callback does slow or blocking work inline; slow work replies later with `gen_server:reply/2`
- ☐ Real timers use `send_after/start_timer`, or the right `gen_statem` timeout type
- ☐ Anything whose handling depends on a mode is a `gen_statem`
- ☐ Values that decide which events can be handled live in the state, not the data
- ☐ No two servers `call` each other
- ☐ High-volume producers have backpressure (`call`, batching or shedding)
- ☐ Servers that need cleanup trap exits (chapter 1, section 4.1)

Sources

- `gen_server` Behaviour, OTP Design Principles: https://www.erlang.org/doc/system/gen_server_concepts.html
- `gen_server` module reference: https://www.erlang.org/doc/apps/stdlib/gen_server.html
- `gen_statem` Behaviour, OTP Design Principles: <https://www.erlang.org/doc/system/statem.html>

6. Distribution

Chapter 6 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why distribution looks easy and isn't

Distributed Erlang makes a remote process look like a local one: the same pids, the same `!`, the same links and monitors. That transparency is the attraction and the trap. The syntax hides the network, but the network still fails in ways that one machine never does.

On one node, a monitored process that stops answering is dead. Across nodes, it might be dead, slow, or cut off by a partition while still running happily on the other side. Every design choice in this chapter comes back to that ambiguity: from inside one node, *unreachable* looks exactly like *dead*.

2. The building blocks

Concept	What it is	Watch out for
Node name	<code>name@host</code> , set with <code>-sname</code> (short) or <code>-name</code> (long)	Short-name and long-name nodes cannot talk to each other
Cookie	Shared secret checked at connection time	Protects against accidents, not attackers
epmd	Local daemon mapping node names to ports	Must be reachable, and must be firewalled
Transitive connect	Connecting to B also connects you to B's peers	On by default; <code>-connect_all false</code> turns it off
Hidden node	Connects without joining the mesh	Not listed by <code>nodes/0</code> ; good for tooling and consoles
<code>global</code>	Cluster-wide name registry and locks	Name clashes after a partition are resolved by killing one process

Security, in the docs' own words

The cookie is not encryption. The docs are blunt: starting a distributed node without `-proto_dist inet_tls` exposes it “to attacks that may give the attacker complete access to the node and by extension the cluster.” Traffic is in clear text by default. Any node that connects can run any code on yours.

Detecting failure

Nodes exchange ticks. With the default `net_ticktime` of 60 seconds, a silent peer is declared down after 45 to 75 seconds. Then every link to a process on that node fires with reason `noconnection`, and every monitor delivers `'DOWN'` with `noconnection`. `net_kernel:monitor_nodes/1` lets a process watch nodes come and go.

3. How the runtime actually does it

One connection per pair of nodes

All traffic between two nodes shares one connection: messages, links, monitors, ticks. Ordering between one sender and one receiver still holds, as in chapter 2. But a large message sits in the same pipe as everything else.

The distribution buffer

Outgoing data queues in a per-connection buffer. When it passes the busy limit (`+zdbb1`, default 1024 KB), every process sending to that node is suspended until the buffer drains. One slow peer or one huge message can freeze senders all over your node.

Signals can be lost

The docs are explicit: “signals can be lost if the distribution channel goes down.” A successful `!` means the message was handed to the runtime, not that it arrived. When the connection drops, whatever was in flight is gone, and the only notice you get is `noconnection` on links and monitors.

Partitions and global

When connections break, `global` (since OTP 25, by default) actively disconnects nodes to avoid overlapping partitions, so the cluster splits into clean, fully connected groups. When groups merge and two of them registered the same global name, the default resolver picks one pid at random and kills the other.

4. Plausible but wrong

4.1 “Node down” means “process dead”

```
handle_info({'DOWN', _, process, _Pid, noconnection}, State) ->
  promote_self_to_leader(),
  {noreply, State}.
```

`noconnection` means the link failed, not that the remote process stopped. During a partition, both sides see each other as down and both promote themselves: two leaders, two writers, divergent data. Fix: treat `noconnection` as “unknown”. For anything that must be single-leader, use a consensus library (such as `ra`, which implements Raft) or an external coordinator, not node monitoring.

4.2 Treating a send as a delivery

```
replicate(Nodes, Update) ->
  [{store, N} ! {apply, Update} || N <- Nodes],
  ok.
```

This returns `ok` whether or not any replica got the update. A connection that drops mid-send silently loses it. Fix: have replicas acknowledge, retry on missing acks, and make updates idempotent (version numbers or unique ids) so retries are safe.

4.3 Shipping a large payload over the cluster link

```
{cache, OtherNode} ! {warm, ets:tab2list(big_cache)}.    %% 500 MB
```

The payload fills the distribution buffer, suspends every other process sending to that node, and delays ticks and control messages behind it. Fix: send in chunks with flow control (the receiver asks for the next chunk), or move bulk data over a separate channel such as a TCP socket or shared storage.

4.4 Sending anonymous funs to other nodes

```
rpc:call(Node, erlang, apply, [fun() -> my_mod:work(Arg) end, []]).
```

A fun is tied to the exact version of the module that created it. If `Node` runs a different build, which is exactly what happens during a rolling upgrade, the call fails with `badfun`. Fix: call by module, function and arguments: `rpc:call(Node, my_mod, work, [Arg])` or `erpc:call(Node, my_mod, work, [Arg])`.

4.5 A cluster on the default cookie

```
erl -name app@10.0.0.5 -setcookie mycookie
```

With ports 4369 (epmd) and the distribution ports open to the network, anyone who guesses or sniffs the cookie can run `os:cmd/1` on your machine. The cookie travels in a challenge that the docs describe as not cryptographically secure. Fix: TLS distribution (`-proto_dist inet_tls`), a firewall that only admits cluster members, and a strong random cookie.

4.6 A global singleton with no plan for merges

```
init([]) ->  
    yes = global:register_name(scheduler, self()),  
    {ok, load_jobs()}.
```

During a partition each side starts its own `scheduler`. When the network heals, `global` kills one at random, along with any jobs only it knew about. Fix: keep the state that matters outside the process (in a replicated store), make startup reload it, and if needed pass your own resolve function to `global:register_name/3`.

4.7 Remote calls with local expectations

```
handle_call(price, _From, State) ->  
    P = gen_server:call({pricing, remote@host}, get),  
    {reply, P, State}.
```

When the remote node hangs rather than crashes, this blocks for the full 5-second call timeout, and it's inside another server's `handle_call`, so that server's callers queue up too. Failure detection itself takes up to 75 seconds. Fix: set explicit short timeouts, avoid remote calls inside callbacks that others wait on, and cache or degrade when the remote side is slow.

5. Review checklist and sources

- ☐ Distribution uses TLS, and epmd plus the distribution ports are firewalled
- ☐ `noconnection` is handled as “unknown”, not “dead”
- ☐ No single-leader logic depends only on node monitoring
- ☐ Cross-node updates are acknowledged and idempotent
- ☐ Bulk data is chunked or sent outside the distribution channel
- ☐ Remote calls use module/function/arguments, not funs
- ☐ Every global name has a plan for what happens after a partition heals
- ☐ Remote calls have explicit timeouts and don't run inside heavily used callbacks

Sources

- Distributed Erlang, Reference Manual: <https://www.erlang.org/doc/system/distributed.html>
- Processes (signals over distribution), Reference Manual: https://www.erlang.org/doc/system/ref_man_processes.html
- global module reference: <https://www.erlang.org/doc/apps/kernel/global.html>
- Kernel application (net_ticktime): https://www.erlang.org/doc/apps/kernel/kernel_app.html
- erl command (+zdbbl): https://www.erlang.org/doc/apps/erts/erl_cmd.html

7. Applications and Releases

Chapter 7 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why applications and releases exist

A supervision tree answers “what happens when a process fails”. An application answers the same question one level up: what happens when a whole component fails, what it needs before it can start, and where its configuration comes from. A release packages a set of applications with a specific runtime into something you can deploy and boot the same way every time.

Much of this lives in metadata files rather than code, which makes it easy to get subtly wrong. A system that works perfectly in `rebar3 shell` can fail to boot as a release, or boot and then run with half its components dead, because the metadata was never tested the way production uses it.

2. The building blocks

The .app file

Each application is described by an `.app` file (generated from `.app.src` by `rebar3`; from `mix.exs` in Elixir):

Key	Meaning
<code>mod</code>	Callback module and argument: <code>{mod, {my_app, []}}</code> calls <code>my_app:start(normal, [])</code>
<code>applications</code>	Applications that must be running before this one starts. Always at least <code>kernel</code> and <code>stdlib</code>
<code>env</code>	Default configuration values
<code>registered</code>	Names this application registers, used to detect clashes
<code>included_applications, start_phases</code>	Advanced: nesting applications and multi-phase startup

A *library application* has no `mod` key. It provides modules but starts no processes.

Start types

What happens when an application’s top supervisor gives up depends on how the application was started:

Type	If it stops normally	If it crashes
<code>permanent</code>	Whole node stops	Whole node stops
<code>transient</code>	Reported, node keeps running	Whole node stops
<code>temporary</code>	Reported, node keeps running	Reported, node keeps running

`application:start/1` and `application:ensure_all_started/1` default to `temporary`. Applications started from a release's boot script are started as `permanent` by default.

Configuration precedence

From highest to lowest priority:

1. Command line: `erl -my_app key value`
2. `sys.config` in the release
3. `env` in the `.app` file

Read with `application:get_env/2,3`. The docs advise against changing configuration with `set_env` in running production systems.

Releases

A release is described by a `.rel` file: a release name and version, an ERTS version, and the exact version of every application. From it, a boot script (`.boot`) is generated that loads and starts every application in dependency order. In practice `rebar3` (via `relx`) or `mix release` builds all of this for you.

3. How the runtime actually does it

The application controller and masters

Starting an application goes through the application controller. It checks that everything in `applications` is already running, then creates an *application master*, which calls your `start/2`. The master becomes the group leader of every process in the application; that's how the system knows which processes belong to it and where their I/O goes.

Your `start/2` must return `{ok, Pid}` where `Pid` is the top supervisor. When that supervisor exits, the application has stopped, and the start type (above) decides what happens next.

Two versions of every module

The runtime can hold two versions of a module at once: *current* and *old*. Loading new code makes the previous version old; both keep running. Loading a third version purges the old one, and any process still running old code is killed.

Which version a process uses depends on how it calls:

- A fully qualified call (`Module:function()` or `?MODULE:loop()`) always goes to current code.
- A local call (`loop()`) stays in whatever version the process is already running.

OTP behaviours handle the switch for you during a release upgrade, calling `code_change/3` to convert state. Hand-written loops do not.

4. Plausible but wrong

4.1 A dependency missing from applications

```
{application, my_app,  
  [{mod, {my_app, []}},  
   {applications, [kernel, stdlib]}]}.    %% but my_app uses jsx and hackney
```

`rebar3 shell` starts every dependency anyway, so everything works in development. In a release, `hackney` may start after `my_app` or not be included at all, and the first HTTP call crashes with `noproc` or `undef`. Fix: list every runtime dependency in `applications`, and test by booting the actual release rather than the shell.

4.2 Configuration read at compile time (Elixir)

```
defmodule MyApp.Client do  
  @timeout Application.get_env(:my_app, :timeout, 5000)  
  def fetch(url), do: HTTP.get(url, timeout: @timeout)  
end
```

A module attribute is evaluated when the module is compiled, so the value is baked into the `.beam`. Changing it in runtime configuration does nothing. Fix: call `Application.get_env/3` inside the function, or use `Application.compile_env/3` when you really do want compile-time values; Elixir then warns if runtime config disagrees. The Erlang equivalent is putting a config value into a `-define` computed by the build.

4.3 `set_env` as a control channel

```
enable_feature(F) ->  
  application:set_env(my_app, F, true).
```

The value is lost on restart, the release's `sys.config` quietly wins again, and any process that read the setting in its `init/1` never sees the change. Fix: keep runtime-changeable settings in an owned store (ETS with a clear owner, or a config service) and restart or notify the processes that depend on them.

4.4 Real work inside `start/2`

```
start(_Type, _Args) ->  
  {ok, _} = my_db:connect(os:getenv("DB_URL")),  
  my_sup:start_link().
```

The whole node's boot waits on the database. If the database is down, `start/2` crashes, the application fails to start, and with a permanent start type the node exits, often into a restart loop driven by `systemd` or `Kubernetes`. Fix: `start/2` should only start the top supervisor; connections belong in supervised workers that retry (chapter 1, section 4.3).

4.5 A hand-written loop and hot code loading

```
loop(State) ->
  receive
    {work, X} -> loop(handle(X, State))
  end.
```

The local call `loop(...)` keeps the process in the version it started with. After one code load it runs old code; after a second load it is killed. Fix: use a behaviour, or make the recursive call `?MODULE:loop(...)` and design the state so old and new versions can both read it.

4.6 A node that keeps running without its application

```
%% custom startup script
main() ->
  {ok, _} = application:ensure_all_started(my_app),
  timer:sleep(infinity).
```

`ensure_all_started/1` uses the `temporary` start type. When `my_app`'s top supervisor exceeds its restart intensity, the application stops and the node carries on, still answering health checks while doing nothing useful. Fix: boot as a proper release (permanent by default), or call `application:ensure_all_started(my_app, permanent)`, so a dead application takes the node down and your orchestrator restarts it.

5. Review checklist and sources

- ☐ Every runtime dependency is listed in `applications`
- ☐ The system is tested by booting the release, not only in the shell
- ☐ Configuration is read at runtime where it is meant to be changeable
- ☐ `set_env` is not used as a runtime control mechanism
- ☐ `start/2` only starts the top supervisor
- ☐ Start types are chosen deliberately; production applications are `permanent`
- ☐ Long-running processes are behaviours, or make fully qualified recursive calls

Sources

- Applications, OTP Design Principles: <https://www.erlang.org/doc/system/applications.html>
- Releases, OTP Design Principles: https://www.erlang.org/doc/system/release_structure.html
- Compilation and Code Loading, Reference Manual: https://www.erlang.org/doc/system/code_loading.html

8. Testing

Chapter 8 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why testing is where AI-written code gets caught

The previous chapters share one theme: code that looks right and passes a happy-path test, then fails under crashes, timing or load. Tests are where that gap gets closed. Only tests that deliberately cause the unhappy paths catch those bugs; tests that re-check the happy path don't.

This matters more when an assistant writes the code, because an assistant asked for tests tends to write tests that mirror the implementation: same assumptions, same blind spots. The tools below are useful exactly to the extent that they check behaviour the author didn't think about.

Erlang has four tools for this, each for a different job:

Tool	Best for	Ships with OTP
EUnit	Fast unit tests of functions and small processes	Yes
Common Test	Integration and system tests, whole applications, multiple nodes	Yes
Dialyzer	Finding type contradictions without running code	Yes
PropEr (Elixir: StreamData)	Generated inputs and randomised state-machine tests	No, a library

2. The building blocks

EUnit

A function ending in `_test()` is a test. A function ending in `_test_()` is a *generator* that returns tests, which is how you attach fixtures. `?assert`, `?assertEqual`, `?assertMatch` and `?assertError` do the checking; their underscore forms (`?_assertEqual`) build test objects for generators.

```
counter_test_() ->
  {foreach,
   fun() -> {ok, P} = counter:start_link(), P end,    %% setup
   fun(P) -> gen_server:stop(P) end,                  %% cleanup
   [fun(P) -> ?_assertEqual(0, counter:get(P)) end,
    fun(P) -> ?_test(begin counter:inc(P),
                      ?assertEqual(1, counter:get(P)) end) end]}.
```

Each test runs in a separate process with a default timeout of 5 seconds.

Common Test

A suite is a module named `*_SUITE`. Configuration callbacks run at each level: `init_per_suite`, `init_per_group` and `init_per_testcase`, each with a matching `end_per_...`. Groups can run test cases in sequence, in parallel or in random order. Common Test is the right place to boot the real application and test it the way it runs in production.

Dialyzer

Dialyzer infers *success typings*: the widest types for which a function could possibly succeed. It reports only contradictions it can prove, so every warning is worth reading. The flip side, as the docs say, is that it “will sometimes not report every bug.” A clean run doesn’t mean correct code.

Property-based testing

Instead of writing examples, you state a property that must hold for all inputs and let the tool generate hundreds of cases. When one fails, the tool *shrinks* it to a minimal counterexample. The stateful variant generates random sequences of API calls against a model, which is the most effective way to find the ordering and restart bugs from earlier chapters.

3. How the runtime actually shapes your tests

Ordering gives you free synchronisation

Chapter 2’s guarantee (messages from one sender to one receiver stay in order) is a testing tool. If your test casts to a server and then calls it, the call is only handled after the cast. So a `call` works as a barrier that proves the cast was processed, with no sleeping.

Links reach into the test

Tests often start servers with `start_link`, which links the server to the test process. If the server crashes, the test process gets an exit signal. Depending on setup, that kills the test with a confusing error, or leaves a server registered under a fixed name that makes the next test fail with `already_started`.

Timeouts can skip cleanup

The EUnit docs warn that in `local` mode, if a timeout fires, “the entire fixture is abruptly terminated (without running the cleanup)”. Leftover processes and tables then leak into later tests.

4. Plausible but wrong

4.1 Sleeping to wait for async work

```
cast_test() ->
  {ok, P} = store:start_link(),
  store:put_async(P, k, v),
```

```
timer:sleep(100),  
?assertEqual({ok, v}, store:get(P, k)).
```

The test passes on a laptop and fails on a loaded CI machine, or it wastes 100 ms every time. Fix: remove the sleep. Because `get` is a `call` from the same process, it is handled after the cast. When the work happens in a different process, wait for a message or a monitor instead.

4.2 Tests that share a registered server

```
a_test() -> {ok, _} = cache:start_link(), ?assert(cache:ping()).  
b_test() -> {ok, _} = cache:start_link(), ?assert(cache:ping()).
```

`cache` registers a name. Whether `b_test` sees `{error, {already_started, _}}` depends on whether the first server has died yet, so the suite passes or fails depending on test order. Fix: a `foreach` fixture that stops the server in cleanup, or servers that accept a name parameter so each test can use its own.

4.3 Only testing examples the author thought of

```
roundtrip_test() ->  
  ?assertEqual(<<"hi">>, decode(encode(<<"hi">>))).
```

It passes, and says nothing about empty input, non-ASCII bytes, or the byte your escape table forgot. Fix: a property over generated input:

```
prop_roundtrip() ->  
  ?FORALL(B, binary(), decode(encode(B)) == B).
```

`PropEr` will find the failing byte and shrink it to the smallest example.

4.4 Never testing the crash

```
worker_test() ->  
  {ok, _} = my_sup:start_link(),  
  ?assertEqual(ok, worker:do(job)).
```

This checks that the tree starts, not that it recovers, and recovery is what supervision is for. Fix: kill the process and check that the system comes back and behaves:

```
restart_test() ->  
  {ok, _} = my_sup:start_link(),  
  Old = whereis(worker),  
  Ref = monitor(process, Old),  
  exit(Old, kill),  
  receive {'DOWN', Ref, process, Old, killed} -> ok end,  
  ?assertEqual(ok, wait_for(fun() -> worker:do(job) end)).
```

Here `wait_for/1` retries a bounded number of times; the restart is asynchronous, so the test must wait for it, not sleep.

4.5 A spec that makes Dialyzer quieter, not safer

```
-spec parse(binary()) -> {ok, map()}.
parse(Bin) ->
  case json:decode(Bin) of
    M when is_map(M) -> {ok, M};
    _ -> {error, not_object}
  end.
```

The spec leaves out `{error, not_object}`. Dialyzer checks only that the spec and the inferred type overlap, and they do, so it accepts the spec and trusts it. Callers are then analysed as if errors can never happen. A caller that *does* handle `{error, _}` gets a “pattern can never match” warning, which invites someone to delete the correct error handling. Fix: write specs that describe what the function really returns, and treat narrowing specs to silence warnings as a code smell.

4.6 Tests that restate the implementation

```
price_test() ->
  ?assertEqual(100 * 0.9, price:discounted(100)).
```

If the formula is wrong, the test is wrong the same way: it was derived from the code, not from the requirement. This is the typical shape of assistant-generated tests. Fix: take expected values from the specification or a worked example (`?assertEqual(90.0, ...)`), and write properties about invariants instead: a discounted price is never negative and never exceeds the original.

5. Review checklist and sources

- ☐ No `timer:sleep/1` used for synchronisation; tests wait on calls, messages or monitors
- ☐ Every test cleans up its processes; no shared registered names between tests
- ☐ Pure functions with wide input spaces have property tests (round-trips, invariants)
- ☐ Supervision behaviour is tested by killing processes and checking recovery
- ☐ Stateful components have at least one stateful property or model-based test
- ☐ Integration tests boot the real application in Common Test
- ☐ Dialyzer runs in CI; specs describe real return values
- ☐ Expected values come from requirements, not from re-running the implementation

Sources

- EUnit User’s Guide: <https://www.erlang.org/doc/apps/eunit/chapter.html>
- Common Test Basics: https://www.erlang.org/doc/apps/common_test/basics_chapter.html
- Dialyzer User’s Guide: https://www.erlang.org/doc/apps/dialyzer/dialyzer_chapter.html
- PropEr: <https://proper-testing.github.io/>
- Fred Hébert, *Property-Based Testing with PropEr, Erlang, and Elixir*: <https://propertesting.com/>

9. Observability

Chapter 9 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why the BEAM is unusually observable

Most runtimes are observed from the outside: logs, OS metrics, a profiler attached after the fact. The BEAM can be observed from the inside while it runs. You can attach a shell to a production node, list every process, read any process's state, trace calls to one function in one process, and detach again, all without a restart.

That power cuts both ways. The same tools that let you inspect a live system can take it down if used carelessly: a trace on a hot function, a query that copies a million-message mailbox, a log line that formats a 100 MB state. This chapter is about seeing clearly without making things worse.

2. The building blocks

Tool	What it tells you	Where it ships
<code>logger</code>	Structured events from your code and from OTP (crashes, supervisor reports)	kernel
<code>sys</code>	State, status, statistics and event traces of one OTP process	stdlib
<code>dbg / trace</code>	Calls, returns, messages and scheduling of chosen processes and functions	runtime_tools / kernel
<code>erlang:memory/0</code> , <code>statistics/1</code> , <code>system_info/1</code>	Node-wide numbers: memory by type, run queues, atom and process counts	erts
<code>erlang:system_monitor/2</code>	Alerts for long GCs, long schedules, huge heaps, busy ports, long message queues	erts
<code>observer</code>	GUI over all of the above	observer
<code>recon</code> (library)	Safe production helpers: top-N processes, rate-limited tracing	external

Logger in brief

Events pass through primary filters, then per-handler filters, then a formatter. There are eight levels, from `emergency` to `debug`. The primary level defaults to `notice`. That means `info` events, including supervisor progress reports, are hidden unless you lower it.

Two ways to log:

```
?LOG_ERROR("file missing: ~ts", [Name]),           %% format string
?LOG_ERROR({event => file_missing, file => Name, reason => enoent}) %% report
```

Prefer reports: they stay structured all the way to the handler, so they can be filtered, indexed and formatted as JSON without parsing strings. Prefer the `?LOG_*` macros over `logger:*` functions: they add location metadata and skip evaluating their arguments when the level is filtered out.

3. How the runtime actually does it

Logger protects itself by dropping

The standard handler watches its own queue. With the defaults, above 10 queued events callers are made to wait (sync mode); above 200 new events are dropped; above 1000 the queue is flushed. Separately, a burst limit allows 500 events per second per handler, and drops the rest until the window ends.

This keeps logging from killing the node, but it means that during an incident, exactly when events come fastest, some of them are thrown away.

sys talks to the process

`sys:get_state/1` and friends work by sending a *system message* to the target and waiting for a reply (default timeout 5000 ms). The process must be responsive to answer. A process that is stuck in a long callback, or has a huge mailbox, will time out. The docs are clear that these functions are “intended only to help with debugging.”

Tracing costs are paid by the traced process

When a traced function is called, the runtime builds a trace message and sends it to the tracer. Tracing a function called 100,000 times per second creates 100,000 messages per second into one tracer process, whose mailbox then grows without bound (chapter 2). Setting trace flags on `all` processes applies the cost everywhere. OTP 27 added trace sessions (`dbg:session_create/1`, the `trace` module) so separate tracing activities no longer interfere with each other.

4. Plausible but wrong

4.1 Paying to format logs nobody sees

```
handle_cast(Msg, State) ->
  logger:debug(io_lib:format("state: ~p", [State])),
  ...
```

`io_lib:format` runs on every call, before `logger` checks the level, even though debug is filtered out in production. With a large state this is the most expensive line in the server. Fix: `?LOG_DEBUG("state: ~p", [State])`, which skips everything when debug is off. For big terms, log selected fields in a report instead of the whole state.

4.2 Logging entire payloads

```
?LOG_ERROR("request failed: ~p", [Request]).    %% Request holds a 20 MB body
```

Formatting a 20 MB term takes real time and memory, then floods the handler into drop mode, and the next few hundred useful events are lost. It can also put personal data in logs. Fix: log identifiers and sizes (`#{request_id => Id, body_bytes => byte_size(Body)}`), and set `depth` and `chars_limit` in the formatter config as a safety net.

4.3 Tracing a hot function on every process

```
dbg:tracer(),
dbg:p(all, c),
dbg:tpl(json, decode, x).
```

If `json:decode` runs thousands of times per second, the tracer's mailbox explodes, the shell becomes unusable, and memory climbs until the node struggles. People have taken down production this way. Fix: trace one process (`dbg:p(Pid, c)`), or use a rate-limited tool such as `recon_trace:calls({json, decode, '_'}, 10)`, which stops by itself after 10 calls. Always end with `dbg:stop()`.

4.4 Reading a mailbox to count it

```
{messages, Msgs} = process_info(Pid, messages),
length(Msgs).
```

This copies every message into your process just to count them. On the process you are investigating, which has a million messages because it is in trouble, that copy can take down the shell or the node. Fix: `process_info(Pid, message_queue_len)`. To find the worst offenders, use `recon:proc_count(message_queue_len, 10)`.

4.5 `sys:get_state` as an API

```
current_config() ->
  #{config := C} = sys:get_state(config_server),
  C.
```

This bypasses the server's interface, depends on its internal state shape, and times out whenever the server is busy. It works in every test and fails in the first overloaded minute of production. Fix: add a real `get_config` call to the server, or publish the value where readers can get it directly (ETS or `persistent_term`, chapter 4).

4.6 Trusting the absence of a log line

```
%% post-incident review
%% "no 'payment_failed' errors in the logs, so no payments failed"
```

During the spike the handler was in drop mode or over its burst limit, so the missing errors may simply have been dropped. Fix: count important events with metrics (a counter per event, as with `telemetry` in Elixir), which do not drop under load, and alert on logger drop counts themselves.

4.7 Watching only OS-level metrics

```
CPU 40%, RSS 3.1 GB, all green
```

The BEAM hides its most useful failure signals from the OS: a growing message queue, run queue length, binary memory (chapter 3), atom count approaching the limit, or long GC pauses. Fix: export `erlang:memory()` by type, `erlang:statistics(total_run_queue_lengths)`, `system_info(atom_count)` and process counts. Install an `erlang:system_monitor/2` handler for `long_gc`, `long_schedule`, `large_heap` and long message queues, so pathologies announce themselves.

5. Review checklist and sources

- ☐ Code logs with `?LOG_*` macros, preferably as structured reports
- ☐ No log line formats whole states, payloads or unbounded terms
- ☐ Formatter `depth` / `chars_limit` are set; logger drop counts are monitored
- ☐ Important events are also counted as metrics
- ☐ Tracing in production is limited to specific processes or is rate-limited, and always stopped
- ☐ Mailbox sizes are read with `message_queue_len`, never `messages`
- ☐ `sys` functions are used only for debugging, never as an API
- ☐ BEAM-level metrics are exported and `system_monitor` is installed

Sources

- Logging, Kernel User's Guide: https://www.erlang.org/doc/apps/kernel/logger_chapter.html
- `sys` module reference: <https://www.erlang.org/doc/apps/stdlib/sys.html>
- `dbg` module reference: https://www.erlang.org/doc/apps/runtime_tools/dbg.html
- `erlang` module reference (`system_monitor`, `memory`, `statistics`):
<https://www.erlang.org/doc/apps/erts/erlang.html>
- recon library: <https://ferd.github.io/recon/>

10. Schedulers and Performance

Chapter 10 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why BEAM performance is about latency first

The BEAM is built to keep many things responsive at once, not to finish one thing as fast as possible. Its schedulers interrupt every process after a small budget of work, so one busy process cannot freeze the others. That fairness is why a node with a million processes still answers promptly, and it shapes what “fast” means here.

Most real performance problems on the BEAM are not slow arithmetic. They are a single process everyone waits on, a mailbox that keeps growing, a native call that won’t give the CPU back, or memory growth that makes garbage collection expensive. The tools and habits in this chapter aim at those.

2. The building blocks

Schedulers

By default the runtime starts one normal scheduler thread per CPU core. Each has its own run queue and moves Erlang processes on and off the core. A process runs until it uses up its budget of *reductions* (roughly, function calls and units of built-in work), blocks in `receive`, or finishes; then the next process gets a turn. This is preemptive multitasking in user space, and it’s why an infinite loop in one process does not stop the others.

Two extra pools exist for work that can’t be interrupted:

Pool	For	Default size
Normal schedulers	All Erlang code	One per core
Dirty CPU schedulers	Long native CPU work	One per core
Dirty I/O schedulers	Long native blocking I/O	10

Profiling tools

The docs are direct: “Even experienced software developers often guess wrong about where the performance bottlenecks are.” Measure first.

Tool	Measures	Overhead
<code>tprof</code>	Call counts, time, or heap allocation per function	Tracing-based; depends on what you measure
<code>eprof</code>	Time per function, per process	Small
<code>cprof</code>	Call counts only	Low

Tool	Measures	Overhead
fprof	Full call-graph timing	Large
perf (Linux, with JIT)	Sampled native profile	Much lower
lcnt	Lock contention inside the runtime	Special build

3. How the runtime actually does it

Where parallelism comes from

Only separate processes run in parallel. One process runs on one scheduler at a time, so all work done inside a single `gen_server` is sequential, however many cores you have. Scaling on the BEAM means splitting work across processes: per request, per key, per partition.

Native code and the 1 ms rule

A NIF runs inside a scheduler thread and cannot be preempted. The docs' guideline is that a well-behaved NIF returns "within 1 millisecond." Longer calls block that scheduler and every process queued on it, and the docs list the effects: "degraded responsiveness," "extreme memory usage, and bad load balancing between schedulers." And "a native function that crashes will crash the whole VM."

The fixes, from simplest to most work:

- mark the NIF dirty (CPU-bound or I/O-bound) so it runs on a dirty scheduler
- split the work into chunks with `enif_schedule_nif`
- report progress with `enif_consume_timeslice` and yield when told to

Classifying CPU-bound work as I/O-bound can starve the normal schedulers, the docs warn.

Benchmarking honestly

The docs recommend making each measurement last several seconds, running each test in a fresh process (reused processes start with larger heaps), and preferring `statistics(runtime)` (CPU time) over `timer:tc` (wall-clock time) when results vary. They also warn that the fastest implementation on one machine is not always the fastest on another.

4. Plausible but wrong

4.1 Benchmarking in the shell

```
1> timer:tc(fun() -> lists:foldl(fun(X, A) -> X + A end, 0, lists:seq(1, 1000000)) end).
```

Funs typed in the shell are interpreted by `erl_eval`, not compiled, so the number is many times slower than the real code. It is also a single short run in a shell process with an already grown heap. Fix: put the

code in a compiled module, run it for seconds in a fresh process, and compare alternatives the same way, or use a benchmarking tool such as `erlperf`.

4.2 Optimising by guessing

```
%% "lists:map is slow, write it by hand"
double([]) -> [];
double([H | T]) -> [H * 2 | double(T)].
```

This is the kind of change an assistant suggests readily. It gains little or nothing, and the real cost is usually elsewhere: a `++` in a loop (chapter 3), a table scan (chapter 4), or a process everybody waits on. Fix: profile first with `tprof` or `perf`, and optimise the functions with the highest *own* time.

4.3 One process doing all the CPU work

```
handle_call({resize, Img}, _From, S) ->
    {reply, image:resize(Img, 800, 600), S}.
```

Every resize in the system goes through one process, so a 64-core machine resizes one image at a time. CPU sits at 1/64 while requests time out. Fix: do the work in the caller or in a pool of workers; keep the server for coordination only.

4.4 A long NIF on a normal scheduler

```
static ERL_NIF_TERM hash_file(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[]) {
    /* reads and hashes a 2 GB file */
}
static ErlNifFunc funcs[] = {{"hash_file", 1, hash_file, 0}};
```

This runs for seconds on a normal scheduler. Every process queued there stalls, including heartbeats and timeouts, so unrelated requests suddenly miss their deadlines. Fix: flag it `ERL_NIF_DIRTY_JOB_IO_BOUND` (it's mostly I/O), or do file work in Erlang and hash chunks in a short NIF.

4.5 CPU work flagged as dirty I/O

```
{"matrix_mul", 2, matrix_mul, ERL_NIF_DIRTY_JOB_IO_BOUND}
```

It works, but there are 10 dirty I/O schedulers by default, typically more than you have cores. Filling them with CPU work competes with the normal schedulers for the cores, and the docs warn this “might starve ordinary schedulers.” Fix: `ERL_NIF_DIRTY_JOB_CPU_BOUND` for anything that computes.

4.6 Reading scheduler spin as load

```
container CPU 95% → autoscaler adds replicas
```

When schedulers run out of work, they briefly busy-wait before sleeping, to pick up new work faster. The OS counts that as CPU use, so a mostly idle node can look busy, and autoscaling on OS CPU scales the wrong way. Fix: measure real scheduler use with `scheduler:utilization/1` (`runtime_tools`). If you share CPUs with other workloads, consider turning busy-waiting down with the `+sbwt` flags.

5. Review checklist and sources

- ☐ Performance claims come from profiles or benchmarks of compiled code, not guesses or shell timings
- ☐ Benchmarks run for seconds in fresh processes
- ☐ CPU-heavy work is spread across processes, not funnelled through one server
- ☐ Every NIF returns within about 1 ms or runs on the correct dirty scheduler
- ☐ CPU-bound native work is flagged dirty CPU, not dirty I/O
- ☐ Capacity decisions use scheduler utilisation, not only OS CPU

Sources

- Profiling, Efficiency Guide: <https://www.erlang.org/doc/system/profiling.html>
- Benchmarking, Efficiency Guide: <https://www.erlang.org/doc/system/benchmarking.html>
- `erl_nif` reference: https://www.erlang.org/doc/apps/erts/erl_nif.html
- `erl` command (scheduler flags): https://www.erlang.org/doc/apps/erts/erl_cmd.html

11. NIFs and Ports

Chapter 11 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why calling native code is a safety decision

Everything in the previous chapters rests on one guarantee: a failure in one process cannot corrupt another. Native code can break that guarantee. The moment C, C++, Rust or Zig runs inside the VM, a single bad pointer can take down every process on the node, and the supervision tree cannot help.

So choosing how to talk to native code is really choosing how much of that guarantee to give up in exchange for speed. The OTP docs are unusually direct about it: because a faulty NIF can cause problems “related to both stability and security it is recommended to use an external Port if possible.”

2. The building blocks

Mechanism	Runs where	If the native code crashes	Cost per call	Use when
Port	Separate OS process, talks over stdin/stdout	Only that OS process dies; the port closes	Highest: serialise and copy through a pipe	Default choice; existing tools and libraries
NIF	Inside a VM scheduler thread	The whole VM crashes	Lowest: a function call	Small, hot, well-tested functions where port overhead matters
Linked-in driver	Inside the VM	The whole VM crashes	Low	Legacy; the docs recommend NIFs instead
C node	Separate program speaking the distribution protocol	Only that program dies	Network round-trip	Native code that behaves like a peer node
Socket	Any process, any machine	Only that program dies	Network round-trip	Services that already exist or live elsewhere

In Elixir, Rustler (Rust) and Zigler (Zig) are the common ways to write NIFs. They remove many memory-safety bugs, but not the scheduling rules: a slow Rust NIF blocks a scheduler just like a slow C one.

Ports in one paragraph

`open_port/2` starts an external program, and the process that opened it becomes its owner. Only the owner can talk to it. Data you send arrives on the program’s stdin; what it writes to stdout arrives as `{Port, {data, D}}` messages. `{packet, N}` adds an N-byte length header so each message arrives whole.

If the owner dies, the port closes, and a correctly written external program sees end-of-file on stdin and exits.

NIFs in one paragraph

A NIF is a C function that replaces an Erlang function in a module. It receives terms in an *environment*, builds result terms in the same environment, and returns. It runs on the calling process's scheduler thread, is never preempted, and should return within about 1 ms or run on a dirty scheduler (chapter 10).

3. How the runtime actually does it

Terms belong to environments

A term passed to a NIF is valid only inside the environment it came in, and only until the NIF returns. The docs say a term “is valid until its environment is destructed.” To keep data between calls, copy it into a process-independent environment (`enif_alloc_env` plus `enif_make_copy`) or keep it in native memory.

Resource objects are how native memory meets the GC

To give Erlang a handle to native memory, allocate a resource (`enif_alloc_resource`), wrap it with `enif_make_resource`, and register a destructor. Erlang code passes the handle around like any term; when the garbage collector finds that nothing references it any more, the runtime calls your destructor. That's the only safe way to tie native lifetimes to Erlang lifetimes.

NIFs run concurrently

The same NIF can run on every scheduler at once. It is thread-safe only if it “acts as a pure function and only reads the supplied arguments.” Any shared native state needs its own locking.

Errors

`enif_make_badarg` makes the NIF raise `badarg` when it returns, whatever else it returns. Anything worse (a segfault, an abort, an out-of-bounds write) is not an exception: it is the end of the node.

4. Plausible but wrong

4.1 A NIF where a port would do

```
/* nif wrapping an image library to generate thumbnails */
static ERL_NIF_TERM thumbnail(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[]);
```

Image decoders parse untrusted input and have a long history of memory-safety bugs. Inside a NIF, one malformed upload crashes the whole node, and every connected user with it. Thumbnails take tens of

milliseconds, so the port overhead is noise. Fix: run the tool as a port (or a pool of ports) under a supervisor; a crash then kills one OS process and restarts it.

4.2 Caching a term across calls

```
static ERL_NIF_TERM cached_config;

static ERL_NIF_TERM set_config(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[]) {
    cached_config = argv[0];          /* belongs to env */
    return enif_make_atom(env, "ok");
}
```

`argv[0]` dies with `env` when the NIF returns. Later calls read freed memory: sometimes the right value, sometimes garbage, sometimes a crash hours later with no obvious cause. Fix: copy it into an environment you own (`enif_make_copy` into an `enif_alloc_env env`), or convert it to a native struct.

4.3 Returning a raw pointer

```
return enif_make_uint64(env, (uint64_t)(uintptr_t) handle);
```

Erlang now holds an integer that looks like a handle. Nothing frees the memory when it's no longer used, a copy can be "closed" twice, and any integer can be passed back in and dereferenced. Fix: a resource object with a destructor; it is freed exactly once, when the garbage collector finds it unreferenced.

4.4 Shared state without locking

```
static int counter = 0;
static ERL_NIF_TERM next_id(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[]) {
    return enif_make_int(env, ++counter);
}
```

Calls from different schedulers run in parallel, so two callers can get the same id, which is a correctness bug that no Erlang test on a single core will reveal. Fix: an atomic increment or `enif_mutex_*`. Better still, keep the counter in Erlang (`ets:update_counter`, chapter 4) and keep the NIF pure.

4.5 A port whose program outlives its owner

```
P = open_port({spawn, "ffmpeg -i in.mp4 out.webm"}, [exit_status]).
```

If the owner crashes, the port closes and the external program's stdin reaches end-of-file. But `ffmpeg` here never reads stdin, so it keeps running, orphaned and invisible to your supervision tree. Repeat the crash and you have dozens of them. Fix: run tools through a small wrapper that exits when stdin closes, or use a process-management library that kills the child when its owner dies.

4.6 Printing debug output on the protocol channel

```
printf("got request of %d bytes\n", len); /* in a {packet, 2} port program */
```

stdout is the data channel. The debug line is read as a length header plus garbage, and the Erlang side gets corrupted messages or a hung protocol. The docs also warn that buffered stdio must not be used for the protocol itself. Fix: log to stderr, and write protocol bytes directly to file descriptor 1.

4.7 Building a shell command from input

```
convert(Filename) ->  
    os:cmd("convert " ++ Filename ++ " out.png").
```

os:cmd/1 runs its argument through a shell. A filename like x.jpg; rm -rf ~ is a command, not a name. Fix: open_port({spawn_executable, Path}, [{args, [Filename, "out.png"]}, exit_status]). The arguments go straight to the program with no shell in between.

4.8 A stream port without framing

```
P = open_port({spawn_executable, Bin}, [binary]),  
port_command(P, term_to_binary(Req)).
```

Without {packet, N} or {line, L}, the port is a byte stream: one message can arrive in pieces, and two can arrive glued together. The code works with small messages on an idle machine and fails with large ones under load. Fix: {packet, 4} on the Erlang side and matching length prefixes in the program, or {line, MaxLen} for text protocols.

5. Review checklist and sources

- ☐ Native code runs as a port unless measurements show the port overhead is unacceptable
- ☐ NIFs never keep terms from a process-bound environment after returning
- ☐ Native memory handed to Erlang is always a resource object with a destructor
- ☐ Shared native state is locked or atomic; ideally NIFs are pure
- ☐ NIFs finish within ~1 ms or are flagged dirty (chapter 10)
- ☐ Port programs exit on stdin end-of-file; orphaned programs are impossible
- ☐ Port programs log to stderr only
- ☐ No shell commands are built from input; spawn_executable with args is used instead
- ☐ Port protocols are framed with {packet, N} or {line, L}

Sources

- Interoperability Overview: <https://www.erlang.org/doc/system/overview.html>
- Ports tutorial: https://www.erlang.org/doc/system/c_port.html
- erl_nif reference: https://www.erlang.org/doc/apps/erts/erl_nif.html
- erlang:open_port/2 reference: <https://www.erlang.org/doc/apps/erts/erlang.html>

12. Elixir Alongside Erlang

Chapter 12 · checked against OTP 29.1.1 and the Elixir docs · Sep 25, 2026

1. Why the previous eleven chapters apply to Elixir unchanged

Elixir compiles to the same BEAM bytecode and runs on the same runtime as Erlang. Processes, mailboxes, binaries, ETS, supervision, distribution and schedulers are not Elixir features with Erlang equivalents; they are the same features. Every runtime behaviour in chapters 1–11 holds exactly in Elixir.

What Elixir adds is a layer of convenience: macros that generate child specs, abstractions like `Task` and `Agent`, and a build tool that handles configuration and releases. Convenience hides defaults, and the defaults are where plausible-but-wrong code comes from. This chapter maps the two languages and points out where Elixir's defaults bite.

2. The building blocks

The map between the two

Concept	Erlang	Elixir
Generic server	<code>gen_server</code>	<code>GenServer</code> (use <code>GenServer</code>)
State machine	<code>gen_statem</code>	<code>:gen_statem</code> directly (no core wrapper)
Supervisor	<code>supervisor</code>	<code>Supervisor</code>
Dynamic children	<code>simple_one_for_one</code>	<code>DynamicSupervisor</code>
One-off work with a result	<code>spawn_monitor</code> + <code>receive</code>	<code>Task.async</code> / <code>Task.await</code>
Supervised one-off work	a <code>simple_one_for_one</code> of workers	<code>Task.Supervisor</code>
State held in a process	a small <code>gen_server</code>	<code>Agent</code>
Process registry	<code>global</code> , <code>pg</code> , or libraries	<code>Registry</code> (local, built on ETS)
Application	<code>.app.src</code> + <code>application</code> callback	<code>mix.exs</code> + <code>Application</code>
Configuration	<code>sys.config</code>	<code>config/*.exs</code> , <code>config/runtime.exs</code>
Releases	<code>rebar3</code> / <code>relx</code>	<code>mix release</code>
Unit tests	<code>EUnit</code> , <code>Common Test</code>	<code>ExUnit</code>
Strings	lists or binaries	binaries (<code>"..."</code>); charlists are <code>~c"..."</code>
Calling the other language	<code>'Elixir.Module':fun()</code>	<code>:module.fun()</code>

What `use GenServer` generates

`use GenServer` defines `child_spec/1` for you, with `restart: :permanent` and `shutdown: 5000` by default. Override them in the `use` line:

```
use GenServer, restart: :temporary, shutdown: 10_000
```

`Supervisor` and `DynamicSupervisor` read this spec, so the defaults decide what happens on crash, exactly as in chapter 1.

A guiding rule from the Elixir docs

“Use processes only to model runtime properties, such as mutable state, concurrency and failures, never for code organization.” A module of pure functions is not improved by wrapping it in a `GenServer`; it only gains a bottleneck.

3. How the runtime actually does it

Task.async is a linked, monitored child

`Task.async` starts a process that is both linked to and monitored by the caller. The docs spell out the consequence: if the caller crashes, the task crashes too, and vice versa. The reply is always sent, so the caller must `Task.await` it (default timeout 5000 ms). This is right when the result is part of the caller’s work, and wrong when a failure in the task should not take the caller down.

Agent functions run inside the agent

The function you pass to `Agent.get/2` or `Agent.update/2` runs in the agent process, not in the caller. The docs describe the trade-off: expensive work inside the agent blocks every other client; moving it to the client means the state may change in the meantime.

Configuration has two times

Anything in `config/config.exs` and friends is evaluated at build time and baked into the release; `config/runtime.exs` is evaluated when the release boots. Module attributes (`@x`) are evaluated at compile time (chapter 7, section 4.2).

4. Plausible but wrong

4.1 Task.async for work that may fail

```
def handle_call({:enrich, user}, _from, state) do
  task = Task.async(fn -> ExternalAPI.fetch_profile(user.id) end)
  {:reply, Task.await(task), state}
end
```

If the external call raises, the linked task takes the `GenServer` down with it, losing its state because of one bad response. Awaiting inside `handle_call` also blocks the server for up to 5 seconds (chapter 5, section 4.1). Fix: `Task.Supervisor.async_nolink/2` and handle `{:exit, reason}` explicitly, or do the call in the client.

4.2 An Agent doing the work

```
Agent.get(Stats, fn events -> compute_percentiles(events) end)
```

`compute_percentiles/1` runs inside the agent. Every other client waits behind it, and a slow computation turns the agent into the bottleneck of the system. Fix: read the data out (`Agent.get(Stats, &&1)`) and compute in the caller. If the data is read-mostly, put it in ETS instead (chapter 4).

4.3 A GenServer used as a namespace

```
defmodule Tax do
  use GenServer
  def calculate(amount), do: GenServer.call(__MODULE__, {:calc, amount})
  def handle_call({:calc, a}, _from, s), do: {:reply, a * 0.2, s}
end
```

There is no state, no concurrency and no failure to model, only a single process every caller now queues behind. This is a very common shape in AI-generated Elixir. Fix: `def calculate(amount), do: amount * 0.2`.

4.4 Per-connection workers left permanent

```
defmodule Conn do
  use GenServer          # restart: :permanent by default
end

DynamicSupervisor.start_child(ConnSup, {Conn, socket})
```

When a client disconnects abnormally, the supervisor restarts the worker with the same, now-closed socket; it crashes again, and repeated crashes can exhaust the supervisor's restart intensity (chapter 1, section 4.5). Fix: `use GenServer, restart: :temporary` for per-connection and per-request processes.

4.5 Atoms from input

```
def handle_params(%{"sort" => field}, _uri, socket) do
  {:noreply, assign(socket, :sort, String.to_atom(field))}
end
```

Every distinct value becomes a permanent atom; enough requests exhaust the atom table and the node goes down (chapter 3, section 4.5). Fix: `String.to_existing_atom/1`, or better, an explicit allow-list:

```
@sortable %{"name" => :name, "date" => :date}
```

```
sort = Map.fetch!(@sortable, field)
```

4.6 Loading everything to process some of it

```
File.read!("events.log")
|> String.split("\n")
|> Enum.filter(&String.contains?(&1, "ERROR"))
|> Enum.take(10)
```

For a 5 GB file this reads everything into memory, splits all of it, and filters all of it just to return ten lines. Fix: `File.stream!/1` with `Stream.filter/2` and `Enum.take/2`, which reads lazily and stops as soon as ten matches are found.

4.7 Runtime secrets in build-time config

```
# config/prod.exs
config :my_app, api_key: System.get_env("API_KEY")
```

This is evaluated when the release is built, on the build machine. The key there is usually missing, so production gets `nil`; or worse, the build machine's key gets baked into the artifact. Fix: read environment variables in `config/runtime.exs`, which runs on the target at boot.

5. Review checklist and sources

- ☐ Processes model state, concurrency or failure; pure logic stays in plain modules
- ☐ `Task.async` only where a task failure should crash the caller; otherwise `Task.Supervisor.async_nolink`
- ☐ No expensive functions passed into `Agent` calls
- ☐ Per-request and per-connection `GenServers` declare `restart: :temporary`
- ☐ No `String.to_atom/1` on external input
- ☐ Large data is processed with `Stream`, not loaded whole with `Enum`
- ☐ Environment-specific values are read in `config/runtime.exs`
- ☐ Every checklist from chapters 1-11 still applies

Sources

- Companion book, *Learning Elixir from Claude*: <https://github.com/ayarodionov/Learning-Elixir-From-Claude>
- GenServer: <https://hexdocs.pm/elixir/GenServer.html>
- Task: <https://hexdocs.pm/elixir/Task.html>
- Agent: <https://hexdocs.pm/elixir/Agent.html>
- DynamicSupervisor: <https://hexdocs.pm/elixir/DynamicSupervisor.html>
- Configuration and releases: <https://hexdocs.pm/elixir/config-and-releases.html>

13. gen_statem in Depth

Chapter 13 · checked against OTP 29.1.1 · example tested on OTP 25 · Sep 25, 2026

1. Why a state machine behaviour

Many processes are really state machines written without admitting it: a connection that is connecting, connected or backing off; a payment that is pending, authorised or settled; a device that is idle, busy or faulted. Written as a `gen_server`, the state hides in a `status` field, and the rules are scattered across `case` expressions in every callback. The question “what can happen in this state?” has no single place to look.

`gen_statem` makes the machine explicit. The docs describe it with one line:

```
\text{State} \times \text{Event} \rightarrow \text{Actions}, \backslash \text{State}'
```

and call the result “mostly like an Event-Driven Mealy machine”: outputs (replies, timers, messages) depend on both the current state and the incoming event. For anyone from control systems this is familiar ground. Draw the transition table first, then the code is a direct translation of it.

Chapter 5 introduced `gen_statem` next to `gen_server`. This chapter works through one complete machine, then covers the details that decide whether a real machine is correct.

2. The building blocks

Callback mode

`callback_mode/0` returns one of two modes, optionally with `state_enter`:

Mode	Callbacks	State can be	Good for
<code>state_functions</code>	One function per state: <code>locked(EventType, Event, Data)</code>	Atoms only	Few simple states, one state at a time
<code>handle_event_function</code>	One <code>handle_event(EventType, Event, State, Data)</code>	Any term	Tuple states, shared handling across states
+ <code>state_enter</code>	Also called with <code>(enter, OldState, ...)</code> on every entry	—	Setup that must happen on each entry

State versus data

- **State** is what the machine *is*: the part that decides which events are valid.
- **Data** is what the machine *has*: counters, buffers, sockets, configuration.

The docs give a precise test: if a value changes which events you can handle, it belongs in the state. Postponed events are retried only when the state changes, so a value that should unblock them must be part of the state.

Actions

A callback returns the next state, the new data, and a list of actions:

Action	Effect
<code>{reply, From, Reply}</code>	Answer a <code>call</code> ; the only action executed immediately
<code>postpone</code>	Keep this event; retry it after the next state change
<code>{next_event, Type, Content}</code>	Handle this event next, before anything in the mailbox
<code>{state_timeout, T, Msg}</code>	Timer tied to the current state; cancelled on state change
<code>{{timeout, Name}, T, Msg}</code>	Named timer; survives state changes; cancel explicitly
<code>{timeout, T, Msg}</code>	Event timer; cancelled by any event
<code>hibernate</code>	Shrink memory before waiting (costly; not after every event)

Timers accept `cancel` instead of a time, `update` to change the message without restarting the clock, and `{abs, true}` for absolute deadlines.

Return shortcuts

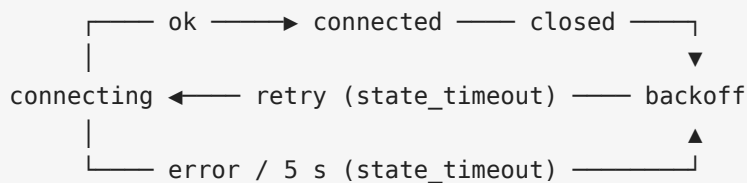
Return	Meaning
<code>{next_state, S, D, Actions}</code>	Go to <code>S</code> (a state change if <code>S != current</code>)
<code>{keep_state, D, Actions}</code>	Same state, new data; no enter call
<code>{repeat_state, D, Actions}</code>	Same state, but run the enter call again
<code>keep_state_and_data</code>	Nothing changes

3. A worked example: a reconnecting client

A client that connects to a server, retries with exponential backoff, and makes callers wait while it is not connected. First, the transition table:

State	Event	Actions	Next state
<code>connecting</code>	<code>enter</code>	<code>start async attempt;</code> <code>state_timeout 5 s</code>	—
<code>connecting</code>	<code>attempt succeeded</code>	<code>reset retries</code>	<code>connected</code>

State	Event	Actions	Next state
connecting	attempt failed or 5 s passed	—	backoff
backoff	enter	state_timeout of 100 ms × 2^retries, max 30 s	—
backoff	timeout	—	connecting
connected	send call	reply	connected
connected	socket closed	—	backoff
any other	send call	postpone	unchanged
any	status call	reply with state	unchanged



The code is a direct translation of the table (also in [examples/conn.erl](#)):

```

-module(conn).
-behaviour(gen_statem).
-export([start_link/1, send/2, status/1]).
-export([callback_mode/0, init/1, handle_event/4]).

start_link(ConnectFun) -> gen_statem:start_link(?MODULE, ConnectFun, []).
send(Pid, Msg)          -> gen_statem:call(Pid, {send, Msg}).
status(Pid)             -> gen_statem:call(Pid, status).

callback_mode() -> [handle_event_function, state_enter].

init(ConnectFun) ->
    {ok, connecting, #{connect => ConnectFun, attempt => undefined,
                        retries => 0, sock => undefined}}.

%% --- connecting: one attempt, at most 5 s ---
handle_event(enter, _Old, connecting, #{connect := F} = D) ->
    Self = self(),
    Ref = make_ref(),
    spawn_link(fun() -> Self ! {connect_result, Ref, F()} end),
    {keep_state, D#{attempt := Ref}, [{state_timeout, 5000, give_up}]};
handle_event(info, {connect_result, Ref, {ok, Sock}}, connecting,
    #{attempt := Ref} = D) ->
    {next_state, connected, D#{sock := Sock, retries := 0}};
handle_event(info, {connect_result, Ref, {error, _}}, connecting,
    #{attempt := Ref} = D) ->
    {next_state, backoff, D};
handle_event(state_timeout, give_up, connecting, D) ->

```

```

    {next_state, backoff, D};

%% --- backoff: wait, doubling the delay each time ---
handle_event(enter, _Old, backoff, #{retries := N} = D) ->
    Delay = min(30000, 100 bsl N),
    {keep_state, D#{retries := N + 1}, [{state_timeout, Delay, retry}]};
handle_event(state_timeout, retry, backoff, D) ->
    {next_state, connecting, D};

%% --- connected ---
handle_event(enter, _Old, connected, _D) ->
    keep_state_and_data;
handle_event({call, From}, {send, Msg}, connected, #{sock := S}) ->
    {keep_state_and_data, [{reply, From, {sent, S, Msg}}]};
handle_event(info, {closed, S}, connected, #{sock := S} = D) ->
    {next_state, backoff, D#{sock := undefined}};

%% --- any state ---
handle_event({call, From}, status, State, _D) ->
    {keep_state_and_data, [{reply, From, State}]};
handle_event({call, _From}, {send, _}, _NotConnected, _D) ->
    {keep_state_and_data, [postpone]};
handle_event(info, {connect_result, _StaleRef, _}, _State, _D) ->
    keep_state_and_data.

```

What each feature is doing:

- **State enter** starts every connection attempt in one place, whether we arrived from `init`, from `backoff`, or after a drop. The initial state gets an `enter` call too.
- `state_timeout` bounds each attempt and each backoff. Leaving the state cancels it automatically, so a successful connect never sees a stale `give_up`.
- `postpone` makes `send` wait while disconnected. On entering `connected` the postponed calls are replayed in order, with no queue in the data and no manual bookkeeping.
- **The attempt reference** ties each result to the attempt that produced it (section 4.1 shows why).

In a test where the first two attempts fail, a `send` issued at startup is answered about 300 ms later: two backoffs of 100 ms and 200 ms, then the third attempt succeeds.

4. How the runtime actually does it

The order of a transition

After a callback returns, the engine does roughly this:

1. Sends every `reply` in the order given.
2. If the state changed and `state_enter` is on, calls the `enter` callback.
3. Postpones the current event if asked.
4. On a state change, puts all postponed events back at the front of the queue, oldest first.
5. Inserts `next_event` events ahead of everything else.

6. Starts or cancels timers. A timer of 0 is not started; its event is queued straight away, after any events already queued.

What an enter call may not do

The enter call is not an event, so the docs forbid changing the state, postponing, inserting events or changing the callback module from it. It can update data, set timers and reply. To leave a state immediately after entering it, set `{state_timeout, 0, Msg}` and handle `Msg`.

No default clause

`gen_server` gives you a default `handle_info` that logs and drops unknown messages. `gen_statem` does not: an event that matches no clause raises `function_clause` and crashes the machine.

Never receive inside a callback

The docs are explicit: a catch-all `receive` inside a `gen_statem` callback can swallow system messages “which in turn can lead to unexpected behaviour.” Anything you would wait for with `receive` should be an event handled by the machine, and waiting should be a state.

5. Plausible but wrong

5.1 Accepting a result from an old attempt

```
handle_event(info, {connect_result, {ok, Sock}}, connecting, D) ->
    {next_state, connected, D#{sock := Sock}}.
```

Attempt 1 times out after 5 s, the machine backs off and starts attempt 2, and then attempt 1's late success arrives. The machine is `connecting`, the pattern matches, and it marks itself `connected` on a socket from an attempt it already gave up on. Fix: tag each attempt with a fresh reference, store it in the data, match on it, and drop stale results explicitly, as in the example.

5.2 A catch-all postpone that captures too much

```
handle_event({call, _, _, State, _} when State /= connected ->
    {keep_state_and_data, [postpone]};
handle_event({call, From}, status, State, _) ->
    {keep_state_and_data, [{reply, From, State}]}.
```

Clauses are tried in order, so `status` is postponed too. A health check calling `status` hangs for exactly as long as the connection is down, which is precisely when you need the answer. Fix: put specific clauses first, and postpone only the events that really need the connection (`{send, _}` in the example).

5.3 Changing state from an enter call

```
handle_event(enter, _Old, connecting, D) ->
  case quick_check() of
    down -> {next_state, backoff, D};
    up    -> {keep_state, D}
  end.
```

Enter calls may not change the state, so the `down` branch crashes the machine with a bad return. Fix: `{keep_state, D, [{state_timeout, 0, down}]}` and handle `state_timeout, down` like any other event.

5.4 A named timer that outlives its state

```
handle_event(enter, _Old, connected, D) ->
  {keep_state, D, [{timeout, heartbeat}, 1000, ping]};
handle_event({timeout, heartbeat}, ping, connected, D) -> ...
```

Named timeouts survive state changes. The connection drops, the machine goes to `backoff`, the heartbeat fires there, and no clause matches: the machine crashes. Fix: use `state_timeout` when the timer belongs to one state; otherwise cancel it on the way out (`{timeout, heartbeat}, cancel`) or handle it in every state.

5.5 No clause for the unexpected

%% handle_event has clauses only for the messages we expect

A stray message (a late `'DOWN'`, a reply to a timed-out request, a typo) raises `function_clause`, and the machine restarts and loses its data. Unlike `gen_server`, there is no default. Fix: a final clause for info events that logs and keeps state. For unexpected calls, reply with an error rather than letting the caller wait.

5.6 Postponing what can never succeed

```
handle_event({call, _}, {send, _}, failed_permanently, _D) ->
  {keep_state_and_data, [postpone]}.
```

The machine never leaves `failed_permanently`, so these calls are never retried. Every caller waits for its full timeout, and the postponed events pile up in the process. Fix: postpone only in states that will eventually be left. Where the answer is known to be “no”, reply `{error, unavailable}` straight away.

5.7 Waiting with receive inside a callback

```
handle_event({call, From}, connect, idle, D) ->
  Sock = start_connect(),
  receive {connected, Sock} -> ok after 5000 -> exit(timeout) end,
  {next_state, connected, D, [{reply, From, ok}]}.
```

For up to 5 seconds the machine cannot answer anything, including system messages from its supervisor and from `sys`. A catch-all variant would swallow them. Fix: the pattern from the example: start the work asynchronously, go to a `connecting` state with a `state_timeout`, and reply when the result event arrives (keep `From` in the data, or postpone the call).

5.8 Data that should be state

```
handle_event({call, _}, {send, _}, connected, #{credit := 0}) ->
    {keep_state_and_data, [postpone]};
handle_event(info, {credit, N}, connected, D) ->
    {keep_state, D#{credit := N}}.
```

Credit arrives but the state is still `connected`, so the postponed sends are never retried (chapter 5, section 4.4). Fix: use a tuple state such as `{connected, Credit}` with `handle_event_function`, or split into `connected` and `blocked`. The docs show exactly this pattern with a complex state.

6. Review checklist and sources

- ☐ The machine has a written transition table, and the code follows it
- ☐ Everything that decides which events are handled lives in the state, not the data
- ☐ Asynchronous results carry a reference to the attempt or request that produced them
- ☐ Specific clauses come before generic `postpone` clauses
- ☐ Enter calls only set up data and timers; immediate transitions use a zero `state_timeout`
- ☐ Per-state timers use `state_timeout`; named timeouts are cancelled or handled in every state
- ☐ A final clause handles unexpected `info` events; unexpected calls get an error reply
- ☐ Events are postponed only in states the machine will eventually leave
- ☐ No `receive` inside callbacks; waiting is modelled as a state

Sources

- `gen_statem` Behaviour, OTP Design Principles: <https://www.erlang.org/doc/system/statem.html>
- `gen_statem` module reference: https://www.erlang.org/doc/apps/stdlib/gen_statem.html

14. Review Cheat Sheet

Chapter 14 · all checklists in one place · Sep 25, 2026

How to use this sheet

This is every chapter's checklist, regrouped by the question a reviewer asks rather than by topic. The number after each item is the chapter that explains it. Use the first section to find suspicious code quickly, and the rest to review it properly.

1. Red flags you can search for

These patterns are not always wrong, but each one deserves a second look.

Search for	Why it's suspicious	Ch.
<code>timer:sleep</code> / <code>Process.sleep</code> in tests	Synchronising by waiting; flaky under load	8
<code>list_to_atom</code> , <code>binary_to_atom</code> , <code>String.to_atom</code>	Atoms from input exhaust the atom table	3, 12
<code>os:cmd</code> with <code>++</code> or string interpolation	Shell injection	11
<code>spawn()</code> without <code>link</code> , <code>monitor</code> or a supervisor	Failures go unnoticed	2
<code>++</code> inside a fold or recursion	Quadratic copying	3
<code><<X/binary, Acc/binary>></code>	Prepending forces a full copy each time	3
<code>ets:tab2list</code> , <code>match_object</code> with <code>'_'</code> keys	Full table scans	4
<code>ets:lookup</code> followed by <code>ets:insert</code> on the same key	Lost updates under concurrency	4
<code>persistent_term:put</code> outside startup code	Global garbage collection on every update	4
<code>{noreply, State, Timeout}</code>	Reset by every message	5
<code>receive</code> inside a <code>gen_statem</code> or <code>gen_server</code> callback	Blocks the machine; can swallow system messages	13
<code>{{timeout, Name}, ...}</code> without a matching cancel	Named timers survive state changes	13
<code>gen_server:call</code> inside <code>handle_call</code>	Blocks the server; possible call cycles	5

Search for	Why it's suspicious	Ch.
<code>catch exit:{timeout, _}</code> followed by a <code>retry</code>	Timeouts are unknown outcomes; duplicates	2
<code>'DOWN'</code> or <code>nodedown</code> handling that takes over leadership	Split brain during partitions	6
Anonymous funs passed to <code>rpc / erpc</code>	<code>badfun</code> across code versions	6
<code>application:set_env</code> outside tests	Not a runtime control channel	7
<code>ensure_all_started(App)</code> in production boot code	Temporary by default; node survives a dead app	7
<code>process_info(Pid, messages)</code>	Copies the entire mailbox	9
<code>sys:get_state</code> outside a shell session	Debug-only, times out under load	9
<code>dbg:p(all, ...)</code>	Traces every process	9
<code>logger:debug(io_lib:format(...))</code>	Formats even when filtered out	9
NIF flags <code>0</code> on long work, or <code>DIRTY_JOB_IO_BOUND</code> on computation	Blocks or starves schedulers	10
<code>static ERL_NIF_TERM</code>	Term kept after its environment died	11
<code>Task.async</code> inside a <code>GenServer</code> callback	Linked crash plus a blocked server	12
<code>@x Application.get_env(...)</code>	Config frozen at compile time	7, 12

2. What happens when something crashes?

- ❑ Child order matches dependency order, and the strategy reflects it (`rest_for_one` where later children need earlier ones) (1)
- ❑ Every child supervisor has `type => supervisor` (1)
- ❑ Per-request and per-connection children are `temporary` (1, 12)
- ❑ `intensity/period` are set deliberately, as burst size and sustained rate, and not identical at every level (1)
- ❑ Work-unit supervisors use `auto_shutdown` and are not `permanent` in their parent (1)
- ❑ Every spawned process is linked, monitored or supervised (2)
- ❑ Every ETS table's owner is chosen deliberately (dedicated process or `heir`) (4)
- ❑ `start/2` only starts the top supervisor; production applications are `permanent` (7)
- ❑ Recovery is tested by killing processes, not only by starting the tree (8)

3. What happens when it shuts down?

- ❑ Workers with cleanup trap exits and have an integer `shutdown` (1, 5)
- ❑ Port programs exit when stdin closes; no orphaned OS processes (11)

4. What happens under load?

- ❑ `init/1` returns fast; slow setup happens in `handle_continue/2` (1)
- ❑ No callback does slow or blocking work inline; slow work replies later with `gen_server:reply/2` (5)
- ❑ Every hand-written `receive` has a catch-all, or is replaced by a behaviour (2, 5)
- ❑ `gen_statem` has a final clause for unexpected `info` events; specific clauses come before generic `postpone` (13)
- ❑ No `receive` inside behaviour callbacks; waiting is modelled as a state (13)
- ❑ High-volume producers have backpressure (`call`, batching or shedding) (5)
- ❑ CPU-heavy work is spread across processes, not funnelled through one server (10)
- ❑ High-traffic processes have `message_queue_data` considered; heap tuning is measured (2)
- ❑ No log line formats whole states, payloads or unbounded terms (9)

5. What happens with concurrent access?

- ❑ No code relies on message ordering except between one sender and one receiver (2)
- ❑ Read-modify-write on ETS uses `update_counter` / `update_element` (4)
- ❑ Traversals of busy tables use `ordered_set`, one ETS call, or `safe_fixtable` (4)
- ❑ Anything whose handling depends on a mode is a `gen_statem`, built from a written transition table (5, 13)
- ❑ Values that decide which `gen_statem` events can be handled live in the state, not the data (5)
- ❑ No two servers `call` each other (5)
- ❑ Shared native state is locked or atomic; ideally NIFs are pure (11)

6. What happens when something is slow or unreachable?

- ❑ Every request/reply is tagged with a fresh reference or monitor (2)
- ❑ Asynchronous results carry a reference to the attempt that produced them; stale results are dropped (13)
- ❑ Timeouts are treated as unknown outcomes; retried operations are idempotent (2, 6)
- ❑ Real timers use `send_after/start_timer` or the right `gen_statem` timeout type (5)
- ❑ Per-state timers use `state_timeout`; named timeouts are cancelled or handled in every state (13)
- ❑ Events are postponed only in states the machine will eventually leave (13)
- ❑ Sends to registered names handle the “not registered right now” case (2)
- ❑ No process holds a sibling’s pid across a possible restart (1)
- ❑ `noconnection` is handled as “unknown”, not “dead” (6)
- ❑ No single-leader logic depends only on node monitoring (6)
- ❑ Cross-node updates are acknowledged and idempotent (6)

- ❑ Every global name has a plan for after a partition heals (6)
- ❑ Remote calls have explicit timeouts and don't run inside heavily used callbacks (6)

7. Where does the memory go?

- ❑ Text in hot paths is binaries or iodata, not character lists (3)
- ❑ Output is built as iodata and written once (3)
- ❑ Binary accumulators only append, and only the newest version is used (3)
- ❑ Slices over 64 bytes of large binaries stored long-term are copied, after checking `referenced_byte_size` (3)
- ❑ Long-lived processes that touch large binaries hibernate or garbage-collect regularly (3)
- ❑ Large shared data is in ETS or passed as binaries, not copied per send (2)
- ❑ ETS objects are small enough to copy on every lookup (4)
- ❑ Bulk data between nodes is chunked or sent outside the distribution channel (6)
- ❑ Large data is streamed, not loaded whole (12)

8. Is it fast for the right reason?

- ❑ Frequent ETS queries use the key; other access paths have a secondary index (4)
- ❑ No `tab2list` or unbound-key `match/select` on large tables in hot paths (4)
- ❑ `bag` tables have no hot keys (4)
- ❑ Concurrency options and `persistent_term` are used for measured reasons (4)
- ❑ Performance claims come from profiles or benchmarks of compiled code (10)
- ❑ Benchmarks run for seconds in fresh processes (10)
- ❑ Capacity decisions use scheduler utilisation, not only OS CPU (10)
- ❑ `+bin_opt_info` has been checked on binary-heavy modules (3)

9. Is native code contained?

- ❑ Native code runs as a port unless port overhead is measured and unacceptable (11)
- ❑ Every NIF returns within about 1 ms or runs on the correct dirty scheduler (10, 11)
- ❑ NIFs never keep terms from a process-bound environment (11)
- ❑ Native memory handed to Erlang is always a resource object (11)
- ❑ Port programs log to stderr only; protocols are framed (11)

10. Is it safe from input and from the network?

- ❑ No atoms are created from external input (3, 12)
- ❑ No shell commands are built from input; `spawn_executable` with `args` instead (11)
- ❑ Distribution uses TLS; `epmd` and distribution ports are firewalled (6)
- ❑ Remote calls use module/function/arguments, not funs (6)

11. Is it configured and deployed correctly?

- ❑ Every runtime dependency is listed in `applications` (7)
- ❑ The system is tested by booting the release, not only in the shell (7)
- ❑ Configuration is read at runtime where it is meant to change; secrets in `config/runtime.exs` (7, 12)
- ❑ `set_env` is not used as a runtime control mechanism (7)
- ❑ Long-running processes are behaviours, or make fully qualified recursive calls (5, 7)

12. Can you see what it's doing?

- ❑ Logging uses `?LOG_*` macros, preferably as structured reports (9)
- ❑ Formatter `depth` / `chars_limit` are set; logger drop counts are monitored (9)
- ❑ Important events are also counted as metrics (9)
- ❑ Production tracing targets specific processes or is rate-limited, and is always stopped (9)
- ❑ Mailbox sizes are read with `message_queue_len` (9)
- ❑ `sys` functions are used only for debugging (9)
- ❑ BEAM-level metrics are exported and `system_monitor` is installed (9)

13. Do the tests prove anything?

- ❑ No sleeping for synchronisation; tests wait on calls, messages or monitors (8)
- ❑ Every test cleans up its processes; no shared registered names (8)
- ❑ Pure functions with wide input spaces have property tests (8)
- ❑ Stateful components have at least one stateful property or model-based test (8)
- ❑ Integration tests boot the real application (8)
- ❑ Dialyzer runs in CI; specs describe real return values (8)
- ❑ Expected values come from requirements, not from the implementation (8)

14. Elixir-specific

- ❑ Processes model state, concurrency or failure; pure logic stays in plain modules (12)
- ❑ `Task.async` only where a task failure should crash the caller; otherwise `Task.Supervisor.async_nolink` (12)
- ❑ No expensive functions passed into `Agent` calls (12)